

Link Github: <https://github.com/MrSampaio/Frush>

Arquivo: \./ViewModel/BookFilterViewModel.swift\

```
swift // // BookFilterViewModel.swift // CH4-Books // // Created by Lucas on 22/08/26. //
```

```
import Foundation import SwiftUI import Combine
```

```
enum BookSortOption: String, CaseIterable { case alphabetical = "Ordem Alfabética" case mostRead = "Mais Lidos (Páginas)" }
```

```
class BookFilterViewModel: ObservableObject { @Published var currentSort: BookSortOption = .alphabetical
```

```
    @Published var selectedCategory: String = "Todos"

    let availableCategories = [
        "Todos", "Romance", "Suspense", "Ação", "Terror",
        "Drama", "Literatura", "Educativo", "Infantil", "Infantojuvenil"
    ]

    func applyFilters(to books: [Books]) -> [Books] {
        var result = books

        if selectedCategory != "Todos" {
            result = result.filter { $0.bookCategory == selectedCategory }
        }

        switch currentSort {
        case .alphabetical:
            result.sort { ($0.bookTitle ?? "") < ($1.bookTitle ?? "") }
        case .mostRead:
            result.sort { $0.bookCurrentPage > $1.bookCurrentPage }
        }

        return result
    }
}
```

```
}
```

Arquivo: \./ViewModel/UserSettingsViewModel.swift\

```
swift // // UserSettingsViewModel.swift // CH4-Books // // Created by Julio Sampaio on 22/08/26. //
```

```
import Foundation import SwiftUI import CoreData import Combine
```

```
class UserSettingsViewModel: ObservableObject {
```

```
@Published var dailyGoal: Int = 0
@Published var lastBookReaded: Books? = nil
@Published var minutesReadToday: Int = 0
```

```
// @Published var dailyGoalMinutes: Int = 15
```

```
// mantém a referência do objeto atual do CoreData para facilitar edições
private var currentSettings: UserSettings?

init(){
    self.fetchUserSettings()
}

func fetchUserSettings() {
    let context = CoreDataManager.shared.viewContext
    let request = NSFetchRequest<UserSettings>(entityName:
    "UserSettings")

    do {
        let results = try context.fetch(request)

        if let settings = results.first {
            self.currentSettings = settings

            let convertedGoal = Int(settings.dailyGoalMinutes)

            self.dailyGoal = convertedGoal
            self.lastBookReaded = settings.lastBookReaded

            let calendar = Calendar.current

            if let lastDate = settings.lastReadingDate,
            calendar.isDateInToday(lastDate) {
                // Se leu hoje, carrega o valor real salvo
                self.minutesReadToday = Int(settings.minutesReadToday)
            } else {
                // Se a última leitura foi em outro dia (ou nunca leu),
                zera na tela e no banco
                self.minutesReadToday = 0
                settings.minutesReadToday = 0
                try? context.save()
            }

            if self.dailyGoal > 0 && self.minutesReadToday <
            self.dailyGoal {
                NotificationManager.shared.scheduleDailyReminders()
            } else {
                NotificationManager.shared.cancelReminders()
            }
        }
    }
}
```

```

        } else {
            // 📌 A MÁGICA ACONTECE AQUI:
            // Se o banco estiver vazio, cria as configurações padrão
            para o app não quebrar mais!
            let newSettings = UserSettings(context: context)
            newSettings.dailyGoalMinutes = 0
            newSettings.minutesReadToday = 0

            try? context.save()

            self.currentSettings = newSettings
            self.dailyGoal = 0
            self.minutesReadToday = 0
            self.lastBookReaded = nil
        }
    } catch let error {
        fatalError("Error when trying to fetch UserSettings data: \(
(error)")
    }
}

func updateUserSettings(newGoal: Int? = nil, newLastBook: Books? = nil)
{
    // Se por algum milagre o currentSettings não estiver carregado,
    força a criação/busca
    if currentSettings == nil {
        fetchUserSettings()
    }

    guard let settings = currentSettings else { return }

    if let newGoal = newGoal {
        settings.dailyGoalMinutes = Int16(newGoal)
        self.dailyGoal = newGoal
    }

    if let newLastBook = newLastBook {
        settings.lastBookReaded = newLastBook
        self.lastBookReaded = newLastBook
    }

    do {
        try CoreDataManager.shared.viewContext.save()
        fetchUserSettings() // Recarrega os dados na ViewModel
        print("UserSetting updated successfully")
    } catch {
        fatalError("Error when trying to update UserSettings: \(
(error)")
    }
}

func saveDailyGoal(minutes: Int) {
    let context = CoreDataManager.shared.viewContext
    let request = NSFetchRequest<UserSettings>(entityName: "UserSettings")

```

```
do {
    let results = try context.fetch(request)

    if let existingSettings = results.first {
        // se já existir configuração, apenas atualiza
        existingSettings.dailyGoalMinutes = Int16(minutes)
    } else {
        // se for a primeira vez, cria o registro
        let newSettings = UserSettings(context: context)
        newSettings.dailyGoalMinutes = Int16(minutes)
    }

    // faz uma função de save separada
    try context.save()

    self.dailyGoal = minutes
    print("Sucess when saving daily goal")

} catch {
    print("Error when saving daily goal: \(
(error.localizedDescription)")
    context.rollback()
}
}

func addCompletedReadingTime(minutes: Int) {

    // faz o fetch caso detecte que o não carregou a currentSettings do
usuário, evita crashes sinistros
    if currentSettings == nil {
        fetchUserSettings()
    }

    guard let settings = currentSettings else { return }

    let calendar = Calendar.current
    let today = Date()

    // verificacao de quando foi a ultima leitura dele
    if let lastDate = settings.lastReadingDate,
!calendar.isDateInToday(lastDate) {
        settings.minutesReadToday = 0
    }

    // soma os minutos atuais com os que já existem
    settings.minutesReadToday += Int16(minutes)
    settings.lastReadingDate = today

    do {
        try CoreDataManager.shared.viewContext.save()
        self.minutesReadToday = Int(settings.minutesReadToday)
        print("Reading updated: \(self.minutesReadToday)")
    }
```

```

        if self.minutesReadToday >= self.dailyGoal {
            NotificationManager.shared.cancelReminders()
        }
    } catch {
        print("Error when updating reading: \(error)")
    }
}

```

```

}

```

```

#if canImport(Uikit) extension View { func hideKeyboard() {
UIApplication.shared.sendAction(#selector(UIResponder.resignFirstResponder), to: nil, from: nil, for: nil) }
} #endif

```

Arquivo: \./ViewModel/StopwatchViewModel.swift\

```

swift // // stopwatchViewModel.swift // CH4-Books // // Created by Lucas on 15/08/26. // import Combine
import Foundation import UIKit

```

```

enum TimerState { case stopped case running case paused }

```

```

class StopwatchViewModel: ObservableObject {

```

```

    @Published var timerState: TimerState = .stopped
    @Published var elapsedTime: TimeInterval = 0
    @Published var totalTime: TimeInterval = 0
    @Published var isRunning: Bool = false
    @Published var timer: Timer? = nil

    @Published var showProgressSheet: Bool = false

    // muda aqui pra puxar a página do banco e ela atualizar o progresso
    dinamicamente

    // controle de Páginas do Livro
    @Published var currentPage: Int16 = 0
    @Published var totalPages: Int16 = 0

    private var cancellables = Set<AnyCancellable>()

    init() {
        // obeservador do tempo de vida ativo do app
        setupAppLifecycleObservers()
        checkPendingSession()
    }

    private func resumeStateOnForeground() {
        checkPendingSession()
    }
}

```

```
private func setupAppLifecycleObservers() {
    // verifica quando o app vai pra segundo plano
    NotificationCenter.default.publisher(for:
UIApplication.didEnterBackgroundNotification)
        .sink { [weak self] _ in
            self?.saveStateOnBackground()
        }
        .store(in: &cancellables)

    // verifica quando fecha o app repentinamente (arrasta pra cima)
    NotificationCenter.default.publisher(for:
UIApplication.willTerminateNotification)
        .sink { [weak self] _ in
            self?.saveStateOnBackground()
        }
        .store(in: &cancellables)

    // verifica quando o app volta para a tela (primeiro plano)
    NotificationCenter.default.publisher(for:
UIApplication.willEnterForegroundNotification)
        .sink { [weak self] _ in
            self?.resumeStateOnForeground()
        }
        .store(in: &cancellables)
}

private func saveStateOnBackground() {
    // só salva caso tenha uma leitura em andamento
    guard timerState == .running || timerState == .paused else { return }

    // salva o exato momento que o app foi fechado/background
    UserDefaults.standard.set(Date().timeIntervalSince1970, forKey:
"savedBackgroundDate")
    // salva o tempo que faltava
    UserDefaults.standard.set(elapsedTime, forKey: "savedElapsedTime")
    // salva o estado do timer (se estava rodando ou pausado)
    UserDefaults.standard.set(timerState == .running ? "running" :
"paused", forKey: "savedTimerState")
}

// função para trazer os estados de volta pro app
private func checkPendingSession() {
    let savedDate = UserDefaults.standard.double(forKey:
"savedBackgroundDate")
    let savedElapsed = UserDefaults.standard.double(forKey:
"savedElapsedTime")
    let savedTotal = UserDefaults.standard.double(forKey: "savedTotalTime")
    let savedStateStr = UserDefaults.standard.string(forKey:
"savedTimerState")

    // se não tiver data salva, significa que não tinha leitura ativa, aí
    ignora
    guard savedDate > 0 else { return }
```

```

    if savedTotal > 0 {
        self.totalTime = savedTotal
    }

    let backgroundDate = Date(timeIntervalSince1970: savedDate)
    let timeAway = Date().timeIntervalSince(backgroundDate) // quanto tempo
o app ficou fora de foco

    if savedStateStr == "running" {
        // subtrai do cronômetro o tempo que o cara passou fora do app
        let newElapsedTime = savedElapsed - timeAway

        if newElapsedTime > 0 {
            // se ainda sobrou tempo, atualiza e volta a rodar o timer
visual
            self.elapsedTime = newElapsedTime
            self.startTimer()
        } else {
            // se o tempo acabou ENQUANTO o app estava fechado
            self.elapsedTime = 0
            self.stop()// ou a sua função que pausa/invalida o timer

            // dispara a bottom sheet para ele preencher as páginas
            DispatchQueue.main.async {
                self.showProgressSheet = true
            }
        }
    } else if savedStateStr == "paused" {
        // se estava pausado, só devolve o tempo exato, não subtrai o tempo
fora
        self.elapsedTime = savedElapsed
        self.timerState = .paused
    }

    // limpa o UserDefaults para não correr o risco de puxar esses dados
numa próxima leitura do zero
    UserDefaults.standard.removeObject(forKey: "savedBackgroundDate")
    UserDefaults.standard.removeObject(forKey: "savedElapsedTime")
    UserDefaults.standard.removeObject(forKey: "savedTotalTime")
    UserDefaults.standard.removeObject(forKey: "savedTimerState")
}

```

```

// // função para trazer os estados de volta pro app // private func resumeStateOnForeground() { // let
savedDate = UserDefaults.standard.double(forKey: "savedBackgroundDate") // let savedElapsed =
UserDefaults.standard.double(forKey: "savedElapsedTime") // let savedStateStr =
UserDefaults.standard.string(forKey: "savedTimerState") // // se não tiver data salva, significa que não
tinha leitura ativa, aí ignora // guard savedDate > 0 else { return } // // let backgroundDate =
Date(timeIntervalSince1970: savedDate) // let timeAway = Date().timeIntervalSince(backgroundDate) //
quanto tempo o app ficou fora de foco // // if savedStateStr == "running" { // // subtrai do cronômetro o

```

```

tempo que o cara passou fora do app // let newElapsedTime = savedElapsed - timeAway // // if
newElapsedTime > 0 { // // se ainda sobrou tempo, atualiza e volta a rodar o timer visual //
self.elapsedTime = newElapsedTime // self.startTimer() // } else { // // se o tempo acabou ENQUANTO o
app estava fechado // self.elapsedTime = 0 // self.stop() // ou a sua função que pausa/invalida o timer //
// // dispara a bottom sheet para ele preencher as páginas // DispatchQueue.main.async { //
self.showProgressSheet = true // } // } // } else if savedStateStr == "paused" { // // se estava pausado, só
devolve o tempo exato, não subtrai o tempo fora // self.elapsedTime = savedElapsed // self.timerState =
.paused // } // // // limpa o UserDefaults para não correr o risco de puxar esses dados numa próxima leitura
do zero // UserDefaults.standard.removeObject(forKey: "savedBackgroundDate") //
UserDefaults.standard.removeObject(forKey: "savedElapsedTime") //
UserDefaults.standard.removeObject(forKey: "savedTimerState") // }

```

```

// progresso do Cronômetro (0.0 a 1.0) para os Anéis
var timeProgress: Double {
    guard totalTime > 0 else { return 0 }
    return (totalTime - elapsedTime) / totalTime
}

// configura a duração inicial (chamado ao rolar o Picker)
func setDuration(_ duration: TimeInterval) {
    self.totalTime = duration
    self.elapsedTime = duration
}

func startTimer() {
    timerState = .running
    isRunning = true

    timer?.invalidate()
    timer = Timer.scheduledTimer(withTimeInterval: 0.1, repeats: true) {
[weak self] _ in
        guard let self = self else { return }
        if self.elapsedTime > 0 {
            self.elapsedTime -= 0.1
        } else {
            self.stop()

            SoundManager.shared.playSound(named: .success)
            DispatchQueue.main.async {
                self.showProgressSheet = true
            }
        }
    }
}

func pauseTimer() {
    timerState = .paused
    isRunning = false
    timer?.invalidate()
}

```

```

        timer = nil
    }

    // para a contagem ao finalizar o tempo
    func stop() {
        timerState = .stopped
        isRunning = false
        timer?.invalidate()
        timer = nil
    }

    // Abandona a leitura e reseta o cronômetro para o valor original
    func abandonTimer() {
        stop()
        elapsedTime = totalTime
    }

    func updatePage(to newPage: Int) {
        self.currentPage = Int16(newPage)
    }

    /*
    func timerFormater() -> String{
        let current = max(0, Int(elapsedTime))
        return String(format: "%02d:%02d", current / 60, current % 60)
    }
    */
    func timerFormater() -> String {
        let current = max(0, Int(elapsedTime))
        let hours = current / 3600
        let minutes = (current % 3600) / 60
        let seconds = current % 60

        if hours > 0 {
            return String(format: "%02d:%02d:%02d", hours, minutes, seconds)
        } else {
            return String(format: "%02d:%02d", minutes, seconds)
        }
    }

    func getCurrentPage(book: Books) -> Int{
        currentPage = book.bookCurrentPage
        return Int(currentPage)
    }

    func getTotalPages(book: Books) -> Int{
        totalPages = book.bookTotalPages
        return Int(totalPages)
    }

```

```

// func getBookProgress(book: Books) -> Int{ // // bookProgress = Double(book.bookCurrentPage) /
Double(book.bookTotalPages) // // return bookProgress //

```

```
// let totalPages = book.bookTotalPages // let currentPage = book.bookCurrentPage // var bookProgress:
Double { // guard totalPages > 0 else { return 0 } // return min(max(Double(currentPage) /
Double(totalPages), 0.0), 1.0) // } //
// return Int(bookProgress) //
// }
```

```
func getBookProgress(book: Books) -> Double {
    let totalPages = book.bookTotalPages
    let currentPage = book.bookCurrentPage

    guard totalPages > 0 else { return 0.0 }
    return min(max(Double(currentPage) / Double(totalPages), 0.0), 1.0)
}
```

```
}
```

Arquivo: \./ViewModel/NotesViewModel.swift\

```
swift // // NotesViewModel.swift // CH4-Books // // Created by Julio Sampaio on 14/08/26. //
```

```
import Foundation import CoreData import Combine import UIKit
```

```
class NotesViewModel: ObservableObject { @Published var savedNotes: [Notes] = []
```

```
enum NoteError: LocalizedError {
    case invalidTitle
    case invalidDescription
    //case invalidCategory
    case invalidBook
    case savingError

    var errorDescription: String? {
        switch self {
            case .invalidTitle:
                return "Insira um título válido."
            case .invalidDescription:
                return "Escreva o conteúdo da nota."
            //case .invalidCategory:
            //    return "Escolha uma categoria para a nota."
            case .invalidBook:
                return "Selecione o livro relacionado a essa nota."
            case .savingError:
                return "Houve um erro ao salvar a nota. Tente novamente."
        }
    }
}
```

```
// mesma lista usada no CategoryMenuView da sheet
let noteCategories = ["Citação", "Resumo", "Pensamento", "Crítica",
```

```
"Conceito", "Lição", "Pergunta", "Favorito"]

init() {
    self.fetchNotes()
}

// funcao que carrega todas as notas do banco e atribui na lista savedNotes
func fetchNotes() {
    let request = NSFetchRequest<Notes>(entityName: "Notes")
    do {
        try self.savedNotes =
CoreDataManager.shared.viewContext.fetch(request)
    } catch let error {
        fatalError("Error when trying to fetch notes data: \(error)")
    }
}

//depois avaliar se realmente e necessaria
func fetchNotes(for book: Books?) {
    guard let book = book else {
        self.savedNotes = []
        return
    }

    let request = NSFetchRequest<Notes>(entityName: "Notes")
    // Filtra para pegar apenas anotações onde o relacionamento 'book' é o
livro atual
    request.predicate = NSPredicate(format: "book == %@", book)

    do {
        self.savedNotes = try
CoreDataManager.shared.viewContext.fetch(request)
    } catch let error {
        print("Erro ao buscar anotações do livro: \(error)")
    }
}

// funcao para salvar notas (chama ela sempre que quer subir efetivamente
para o banco)
func saveNote() throws {
    do {
        try CoreDataManager.shared.viewContext.save()
    } catch let error {
        CoreDataManager.shared.viewContext.rollback()
        print("Error when trying to save new note: \(error)")
        throw NoteError.savingError
    }
}

// funcao que adiciona notas com os parametros a serem recebidos pela view
func addNote(noteTitle: String, noteDescription: String, noteCategory:
String, notePhotos: [UIImage], to book: Books?) throws {

    let cleanTitle = noteTitle.trimmingCharacters(in:
```

```

.whitespacesAndNewlines)
    let cleanDescription = noteDescription.trimmingCharacters(in:
.whitespacesAndNewlines)

    if cleanTitle.isEmpty { throw NoteError.invalidTitle }
    if cleanDescription.isEmpty { throw NoteError.invalidDescription }
    //if noteCategory.isEmpty { throw NoteError.invalidCategory }
    guard let book else { throw NoteError.invalidBook }

    let newNote = Notes(context: CoreDataManager.shared.viewContext)
    newNote.noteTitle = cleanTitle
    newNote.noteDescription = cleanDescription
    newNote.noteCategory = noteCategory
    newNote.book = book

    var photosdataArray: [Data] = []
    for photo in notePhotos {
        if let data = photo.jpegData(compressionQuality: 0.8) {
            photosdataArray.append(data)
        }
    }

    newNote.notePhoto = photosdataArray as NSObject

    try self.saveNote()

    self.fetchNotes()
}

func updateNote(note: Notes, noteTitle: String, noteDescription: String,
notePhotos: [UIImage]) throws {

```

```
// let totalPagesInt = Int16(bookTotalPages) ?? 0
```

```
// let defaultImageData = UIImage(named: "defaultBook")?.jpegData(compressionQuality: 1) ?? Data() //
```

```
// let coverData = bookCover?.jpegData(compressionQuality: 1) ?? defaultImageData
```

```

    let finalTitle = (noteTitle.trimmingCharacters(in:
.whitespacesAndNewlines))
    let finalDescription = (noteDescription.trimmingCharacters(in:
.whitespacesAndNewlines))

    if finalTitle.isEmpty {
        throw NoteError.invalidTitle
    }

    if finalDescription.isEmpty {
        throw NoteError.invalidDescription
    }

    if(note.noteTitle != finalTitle){
        note.noteTitle = finalTitle
    }

```

```

    }

    if(note.noteDescription != noteDescription){
        note.noteDescription = noteDescription
    }

    var photosDataArray: [Data] = []
    for photo in notePhotos {
        if let data = photo.jpegData(compressionQuality: 0.8) {
            photosDataArray.append(data)
        }
    }

    if(note.notePhoto != photosDataArray as NSObject){
        note.notePhoto = photosDataArray as NSObject
    }

```

```
// if(book.bookCover != coverData){ // book.bookCover = coverData // }
```

```

        try self.saveNote()
        self.fetchNotes()
    }

    // funcao para deletar notas
    func deleteNote(note: Notes) throws{
        CoreDataManager.shared.viewContext.delete(note)
        self.savedNotes.removeAll(where: { $0.objectID == note.objectID })

        try self.saveNote()
        self.fetchNotes()
    }

```

```
}
```

```

// private func savePhotoToDisk(_ image: UIImage?) -> String? { // guard let image, let data =
image.jpegData(compressionQuality: 0.8) else { return nil } //
// let fileName = "(UUID().uuidString).jpg" // let url = FileManager.default // .urls(for: .documentDirectory,
in: .userDomainMask)[0] // .appendingPathComponent(fileName) //
// do { // try data.write(to: url) // return fileName // } catch { // print("Erro ao salvar imagem no disco:
(error)") // return nil // } // }

```

Arquivo: \./ViewModel/PhotoLibraryViewModel.swift\

swift // // PhotoLibraryViewModel.swift // CH4-Books // // Created by Julio Sampaio on 14/08/26. //

import Foundation import SwiftUI import Combine import PhotosUI import UIKit import CoreData

```
struct SelectableImage: Identifiable { let id = UUID() let image: UIImage }
```

```
class PhotoLibraryViewModel: ObservableObject { @Published var selectedItem: PhotosPickerItem? = nil
{ didSet { Task { await loadImage() } } }
```

```
    //@Published var selectedImage: UIImage? = UIImage(named: "defaultBook")

    @Published var selectedCoverImage: UIImage? = UIImage(named: "defaultBook")
    ?? UIImage()

    @Published var selectedImage: UIImage? = nil

    @Published var noteImages: [SelectableImage] = []

    private let maxNoteImages = 3

    func loadImage() async {
        guard let item = selectedItem else { return }
        if let data = try? await item.loadTransferable(type: Data.self),
            let image = UIImage(data: data) {
            DispatchQueue.main.async {
                self.selectedCoverImage = image
            }
        }
    }

    func loadExistingImages(_ images: [UIImage]) {
        for image in images.prefix(maxNoteImages) {
            noteImages.append(SelectableImage(image: image))
        }
    }

    func saveImageToCoreData(image: UIImage){
        let newPhoto = Books(context: CoreDataManager.shared.viewContext)

        let defaultImageData = UIImage(named:
"defaultBook")?.jpegData(compressionQuality: 1) ?? Data()
        let imageData = image.jpegData(compressionQuality: 1) ??
defaultImageData

        newPhoto.bookCover = imageData

        do {
            try CoreDataManager.shared.viewContext.save()
            print("Success when saving the book cover")
        } catch let error {
            print("Error when saving the book cover \(error)")
        }
    }

    func convertImageToData(image: UIImage) -> Data?{

        let defaultImageData = UIImage(named:
```

```

"defaultBook"?).jpegData(compressionQuality: 1) ?? Data()
    let imageData = image.jpegData(compressionQuality: 1) ??
defaultImageData

    return imageData
}

func getCoverImage(for book: Books) -> UIImage? {
    if book.entity.attributesByName.keys.contains("bookCover") {

        if let imageData = book.value(forKey: "bookCover") as? Data, let
uiImage = UIImage(data: imageData) {
            return uiImage
        }

        if let imageName = book.value(forKey: "bookCover") as? String,
!imageName.isEmpty, let uiImage = UIImage(named: imageName) {
            return uiImage
        }
    }

    if book.entity.attributesByName.keys.contains("bookImage") {
        if let imageData = book.value(forKey: "bookImage") as? Data, let
uiImage = UIImage(data: imageData) {
            return uiImage
        }
    }
    return nil
}

func loadPickedItems(_ items: [PhotosPickerItem]) {
    Task {
        for item in items {
            guard noteImages.count < maxNoteImages else { break }

            guard let data = try? await item.loadTransferable(type:
Data.self) else { continue }
            guard let uiImage = UIImage(data: data) else { continue }

            await MainActor.run {
                noteImages.append(SelectableImage(image: uiImage))
            }
        }
    }
}

func appendCapturedImage(_ image: UIImage?) {
    guard let image, noteImages.count < maxNoteImages else { return }
    noteImages.append(SelectableImage(image: image))
}

func removeNoteImage(id: UUID) {
    noteImages.removeAll { $0.id == id }
}

```

```
func resetNoteImages() {
    noteImages.removeAll()
}
```

```
}
```

Arquivo: \./ViewModel/BooksViewModel.swift\

```
swift // // BooksViewModel.swift // CH4-Books // // Created by Julio Sampaio on 13/08/26. // import
Foundation import SwiftUI import CoreData import Combine import PhotosUI
```

```
class BooksViewModel: ObservableObject {
```

```
// @EnvironmentObject var photoLibraryViewModel: PhotoLibraryViewModel @Published var savedBooks:
[Books] = []
```

```
enum BookError: LocalizedError {
    case invalidTitle
    case invalidAuthor
    case invalidTotalPages
    case invalidCurrentPage
    //case invalidGoal
    case invalidPageLogic
    case savingError

    var errorDescription: String? {
        switch self {
            case .invalidTitle:
                return "Insira um título válido."
            case .invalidAuthor:
                return "Insira um autor válido."
            case .invalidTotalPages:
                return "Número de páginas inválido."
            case .invalidCurrentPage:
                return "Página atual inválida."
        }
    }
}
```

```
// case .invalidGoal: // return "Escolha uma meta de leitura." case .invalidPageLogic: return "Página atual
deve ser menor que o total de páginas." case .savingError: return "Houve um erro ao salvar o livro. Tente
novamente."
```

```
    }
}

let goalOptions = ["5", "10", "15", "20", "30", "45", "60"]

let bookCategories = ["Romance", "Suspense", "Ação", "Terror", "Drama",
```

```
"Literatura", "Educativo", "Infantil", "Infantojuvenil"]

init(){
    self.fetchBooks()
}

func countGeralReadedPages() -> Int16{
    var totalReadedPages: Int16 = 0
    for book in self.savedBooks{
        totalReadedPages += book.bookCurrentPage
    }

    return totalReadedPages
}

func countBookReadedPages(book: Books) -> Int16 {
    let pagesReaded = book.bookCurrentPage
    return pagesReaded
}

// funcao que carrega todos os livros do banco e atribui na lista books
func fetchBooks(){
    let request = NSFetchRequest<Books>(entityName: "Books")
    do{
        try self.savedBooks =
CoreDataManager.shared.viewContext.fetch(request)
        countGeralReadedPages()

    } catch let error{
        fatalError("Error when trying to fetch books data: \(error)")
    }
}

// funcao para salvar livros (chama ela sempre que quer subir efetivamente
para o banco)
func saveBook() throws{
    do{
        try CoreDataManager.shared.viewContext.save()
    } catch let error{
        CoreDataManager.shared.viewContext.rollback()
        print("Error when trying to save new book: \(error)")
        throw BookError.savingError
    }
}

// funcao que adiciona livros com os parametros a serem recebidos pela view
func addBook(bookTitle: String, bookAuthor: String, bookCover: UIImage?,
bookCategory: String, bookTotalPages: String) throws{

    let totalPagesInt = Int16(bookTotalPages) ?? 0
```

```
// let currentePageInt = Int16(bookCurrentPage) ?? 0 //let lastPageInt = Int16(bookLastPage) ?? 0
```

```

    let cleanTitle = bookTitle.trimmingCharacters(in:
.whitespacesAndNewlines)
    let cleanAuthor = bookAuthor.trimmingCharacters(in:
.whitespacesAndNewlines)

    if cleanTitle.isEmpty {
        throw BookError.invalidTitle
    }

    if cleanAuthor.isEmpty {
        throw BookError.invalidAuthor
    }

    if totalPagesInt <= 0 {
        throw BookError.invalidTotalPages
    }

    let defaultImageData = UIImage(named:
"defaultBook")?.jpegData(compressionQuality: 1) ?? Data()
    let coverData = bookCover?.jpegData(compressionQuality: 1) ??
defaultImageData

    let newBook = Books(context: CoreDataManager.shared.viewContext)
    newBook.bookTitle = cleanTitle
    newBook.bookAuthor = cleanAuthor.isEmpty ? "Desconhecido" : cleanAuthor
    newBook.bookCover = coverData
    newBook.bookCategory = bookCategory.isEmpty ? "Sem categoria" :
bookCategory
    newBook.bookTotalPages = totalPagesInt

```

```
// newBook.bookCurrentPage = bookCurrentPage // newBook.bookGoal = bookGoal
```

```
newBook.isTimerRunning = false newBook.wasLastPageAdded = true
```

```
// if currentePageInt <= 0 { // throw BookError.invalidCurrentPage // }
```

```
// if bookCurrentPage < 0{ // throw BookError.invalidCurrentPage // } //
```

```
// if bookCurrentPage > bookTotalPages { // throw BookError.invalidPageLogic // }
```

```
// if bookGoal <= 0{ // throw BookError.invalidGoal // }
```

```

    try self.saveBook()

    self.fetchBooks()
}

// funcao para deletar livros
func deleteBook(book: Books) throws{

```

```
// guard let index = indexSet.first else { return } // let book = self.savedBooks[index]
```

```
CoreDataManager.shared.viewContext.delete(book)
self.savedBooks.removeAll(where: { $0.objectID == book.objectID })

try self.saveBook()
self.fetchBooks()

}

func updateBook(book: Books, bookTitle: String, bookAuthor: String,
bookCover: UIImage?, bookCategory: String, bookTotalPages: String) throws {
```

```
// guard let index = IndexSet.first else { return } //
// let book = self.savedBooks[index]
```

```
let totalPagesInt = Int16(bookTotalPages) ?? 0

let finalTitle = (bookTitle.trimmingCharacters(in:
.whitespacesAndNewlines))

let defaultImageData = UIImage(named:
"defaultBook")?.jpegData(compressionQuality: 1) ?? Data()

let coverData = bookCover?.jpegData(compressionQuality: 1) ??
defaultImageData

if finalTitle.isEmpty {
    throw BookError.invalidTitle
}

if totalPagesInt <= 0 {
    throw BookError.invalidTotalPages
}

//let finalTotalPages = bookTotalPages ?? book.bookTotalPages
```

```
// let finalCurrentPage = bookCurrentPage ?? book.bookCurrentPage // let finalGoal = bookGoal ??
book.bookGoal
```

```
// if finalTotalPages <= 0 { // throw BookError.invalidTotalPages // }
```

```
// if finalCurrentPage < 0 { // throw BookError.invalidCurrentPage // } //
```

```
// if finalCurrentPage > finalTotalPages { // throw BookError.invalidPageLogic // }
```

```
// if finalGoal <= 0 { // throw BookError.invalidGoal // }
```

```
// só aplica as mudanças se passou em todas as validações
```

```
// book.id = UUID()
```

```
    if(book.bookTitle != finalTitle){
        book.bookTitle = finalTitle
    }

    if(book.bookAuthor != bookAuthor){
        book.bookAuthor = bookAuthor
    }

    if(book.bookCategory != bookCategory){
        book.bookCategory = bookCategory
    }

    if(book.bookTotalPages != totalPagesInt){
        book.bookTotalPages = totalPagesInt
    }

    if(book.bookCover != coverData){
        book.bookCover = coverData
    }

    try self.saveBook()
    self.fetchBooks()
}

@Published var dailyGoalMinutes: Int = 0

func fetchDailyGoal() {
    let request = NSFetchRequest<UserSettings>(entityName: "UserSettings")

    do {
        let results = try CoreDataManager.shared.viewContext.fetch(request)
        if let settings = results.first {
            self.dailyGoalMinutes = Int(settings.dailyGoalMinutes)
        }
    } catch {
        print("Error when fetching daily goal: \(error)")
    }
}

func saveDailyGoal(minutes: Int) {
    let context = CoreDataManager.shared.viewContext
    let request = NSFetchRequest<UserSettings>(entityName: "UserSettings")

    do {
        let results = try context.fetch(request)
```

```

        if let existingSettings = results.first {
            // se já existir configuração, apenas atualiza
            existingSettings.dailyGoalMinutes = Int16(minutes)
        } else {
            // se for a primeira vez, cria o registro
            let newSettings = UserSettings(context: context)
            newSettings.dailyGoalMinutes = Int16(minutes)
        }

        try context.save()

        self.dailyGoalMinutes = minutes

    } catch {
        print("Error when saving daily goal: \
(error.localizedDescription)")
        context.rollback()
    }
}

func updateCurrentPage(book: Books, currentPage: String) throws{
    let convertedCurrentPage = Int16(currentPage) ?? 0

    if convertedCurrentPage == 0 {
        throw BookError.invalidCurrentPage
    }

    book.bookCurrentPage = convertedCurrentPage

    try self.saveBook()
    self.fetchBooks()
}

```

```

// func saveDailyGoal(minutes: Int) { // let context = CoreDataManager.shared.viewContext // let request =
NSFetchRequest(entityName: "Database") //
// do { // let results = try context.fetch(request) //
// if let existingSettings = results.first { // existingSettings.dailyGoalMinutes = Int32(minutes) // } else { //
let newSettings = UserSettings(context: context) // newSettings.dailyGoalMinutes = Int32(minutes) // } //
// // 2. Salve as alterações no Core Data // try context.save() // print("Meta diária de (minutes) minutos
salva com sucesso!") //
// } catch { // print("Erro ao salvar meta diária: (error.localizedDescription)") // } // }

```

Arquivo: \./CH4_BooksApp.swift\

```
swift // // CH4_BooksApp.swift // CH4-Books // // Created by Julio Sampaio on 13/08/26. //
```

```
import SwiftUI import CoreData
```

```
@main struct CH4_BooksApp: App { let persistenceController = CoreDataManager.shared @StateObject
var photoViewModel = PhotoLibraryViewModel() @StateObject var booksViewModel = BooksViewModel()

```

```
@StateObject var notesViewModel = NotesViewModel() @StateObject var stopWatchViewModel =
StopwatchViewModel() @StateObject var userSettingsViewModel = UserSettingsViewModel()
@StateObject var filterViewModel = BookFilterViewModel()
```

```
var body: some Scene {
    WindowGroup {
        RootFlowView()
            .environment(\.managedObjectContext,
persistenceController.viewContext)
            .environmentObject(userSettingsViewModel)
            .environmentObject(photoViewModel)
            .environmentObject(booksViewModel)
            .environmentObject(notesViewModel)
            .environmentObject(stopWatchViewModel)
            .environmentObject(filterViewModel)
    }
}
```

```
}
```

```
struct RootFlowView: View {
```

```
enum Screen {
    case splash
    case firstOnboard
    case secondOnboard
    case main
}

@AppStorage("hasCompletedOnboarding") private var hasCompletedOnboarding =
false

@State private var screen: Screen = .splash

var body: some View {
    ZStack {
        switch screen {
            case .splash:
                SplashView()
                    .transition(.opacity)
                    .task {
                        try? await Task.sleep(for: .seconds(1.2))
                        withAnimation(.easeInOut(duration: 0.35)) {
                            screen = hasCompletedOnboarding ? .main :
.firstOnboard
                        }
                    }

            case .firstOnboard:
                FirstOnboardView {
```

```

        withAnimation(.easeInOut(duration: 0.35)) { screen =
        .secondOnboard }
    }
    .transition(.opacity)

    case .secondOnboard:
        SecondOnboardView {
            hasCompletedOnboarding = true
            withAnimation(.easeInOut(duration: 0.35)) { screen = .main
        }
        }
        .transition(.opacity)

    case .main:
        ContentView()
        .transition(.opacity)
    }
}
}

```

```

}

```

Arquivo: \./Managers/CoreDataManager.swift\

swift // // CoreDataManager.swift // CH4-Books // // Created by Julio Sampaio on 13/08/26. //

import Foundation import CoreData

class CoreDataManager{ let persistentContainer: NSPersistentContainer static let shared = CoreDataManager()

```

var viewContext: NSManagedObjectContext{
    return self.persistentContainer.viewContext
}

init(){
    self.persistentContainer = NSPersistentContainer(name: "Database")
    self.persistentContainer.loadPersistentStores { (description, error) in
        if let error = error{
            print("Error loading persistent stores: \(error)")
        }
    }

    persistentContainer.viewContext.automaticallyMergesChangesFromParent = true

    // atualiza automaticamente os possíveis novos atributos do banco
    let description = persistentContainer.persistentStoreDescriptions.first
    description?.shouldInferMappingModelAutomatically = true

```

```
description?.shouldMigrateStoreAutomatically = true
}
```

```
}
```

```
// // CoreDataManager.swift // CH4-Books // // Created by Julio Sampaio on 13/08/26. //
```

```
//import Foundation //import CoreData // //class CoreDataManager { // let persistentContainer:
NSPersistentContainer // static let shared = CoreDataManager() //
// var viewContext: NSManagedObjectContext { // return self.persistentContainer.viewContext // } //
// init() { // self.persistentContainer = NSPersistentContainer(name: "Database") //
// // Configurar para usar memória RAM // let description = NSPersistentStoreDescription() //
description.type = NSInMemoryStoreType // ← Aqui é a mágica //
self.persistentContainer.persistentStoreDescriptions = [description] //
// self.persistentContainer.loadPersistentStores { (description, error) in // if let error = error { // print("Error
loading persistent stores: (error)") // } else { // print("Core Data loaded in memory ✅ ") // } // } //
// persistentContainer.viewContext.automaticallyMergesChangesFromParent = true // } //}
```

Arquivo: \./Managers/SoundManager.swift\

```
swift // // SoundManager.swift // CH4-Books // // Created by Julio Sampaio on 23/08/26. //
```

```
import Foundation import AVFoundation
```

```
class SoundManager { static let shared = SoundManager() private var audioPlayer: AVAudioPlayer?
```

```
enum SoundType {
    case splash
    case success

    var fileName: (name: String, extension: String) {
        switch self {
            case .splash: return ("splash", "mp3")
            case .success: return ("readSucess", "mp3")
        }
    }
}

func playSound(named sound: SoundType) {
    let file = sound.fileName

    guard let url = Bundle.main.url(forResource: file.name, withExtension:
file.extension) else {
        print("File '\(file.name).\\(file.extension)' not founded")
        return
    }

    do {
```

```

        audioPlayer = try AVAudioPlayer(contentsOf: url)
        audioPlayer?.prepareToPlay()
        audioPlayer?.play()
    } catch {
        print("Error when reproducing sound: \(
(error.localizedDescription)")
    }
}

```

```

}

```

Arquivo: \./Managers/NotificationManager.swift\

swift // // NotificationManager.swift // CH4-Books // // Created by Julio Sampaio on 23/08/26. //

import Foundation import Foundation import UserNotifications

class NotificationManager { static let shared = NotificationManager()

```

private init() {}

func requestPermission() {
    UNUserNotificationCenter.current().requestAuthorization(options:
[.alert, .badge, .sound]) { granted, error in
        if granted {
            print("Permissão de notificação concedida!")
        } else if let error = error {
            print("Erro ao pedir permissão de notificação: \(error)")
        }
    }
}

func scheduleDailyReminders() {
    cancelReminders()

    scheduleNotification(
        id: "morning_reminder",
        hour: 8,
        title: "Bom dia, leitor(a)!",
        body: "Que tal começar o dia avançando algumas páginas do seu
livro?"
    )

    scheduleNotification(
        id: "afternoon_reminder",
        hour: 15,
        title: "Pausa para leitura",
        body: "Tire uns minutinhos da sua tarde para relaxar e progredir na
sua meta de hoje."
    )
}

```

```

        scheduleNotification(
            id: "night_reminder",
            hour: 21,
            title: "O dia está acabando...",
            body: "Ainda dá tempo! Pegue seu livro e relaxe lendo um pouco
antes de dormir."
        )
    }

    // função auxiliar que cria o gatilho da hora exata
    private func scheduleNotification(id: String, hour: Int, title: String,
body: String) {
        let content = UNMutableNotificationContent()
        content.title = title
        content.body = body
        content.sound = .default

        var dateComponents = DateComponents()
        dateComponents.hour = hour
        dateComponents.minute = 0

        // repeats: true faz com que a notificação se repita todo dia nesse
horário (se não for cancelada)
        let trigger = UNCalendarNotificationTrigger(dateMatching:
dateComponents, repeats: true)
        let request = UNNotificationRequest(identifier: id, content: content,
trigger: trigger)

        UNUserNotificationCenter.current().add(request) { error in
            if let error = error {
                print("Error when scheduling notification \(id): \(
(error.localizedDescription)")
            }
        }
    }

    func cancelReminders() {

        UNUserNotificationCenter.current().removeAllPendingNotificationRequests()
    }
}

```

Arquivo: \./Extensions/FontTypography.swift\

swift // // FontTypography.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on 15/08/26. //

import SwiftUI

```
extension Font { enum BitterWeight: String { case regular = "Bitter-Regular" case medium = "Bitter-Medium" case bold = "Bitter-Bold" case semibold = "Bitter-SemiBold" }
```

```
// aplica a fonte Bitter com peso específico ajustada ao Dynamic Type
static func bitter(_ weight: BitterWeight = .medium, style: TextStyle) ->
Font {
    let uiStyle: UIFont.TextStyle = switch style {
    case .largeTitle: .largeTitle
    case .title: .title1
    case .title2: .title2
    case .title3: .title3
    case .headline: .headline
    case .subheadline: .subheadline
    case .body: .body
    case .callout: .callout
    case .footnote: .footnote
    case .caption: .caption1
    case .caption2: .caption2
    @unknown default: .body
    }

    let pointSize = UIFont.preferredFont(forTextStyle: uiStyle).pointSize
    return Font.custom(weight.rawValue, size: pointSize, relativeTo: style)
}
```

```
}
```

Arquivo: \./View/PreviewHelper/PreviewProviderHelper.swift\

```
swift // // test.swift // CH4-Books //
```

```
import Foundation import SwiftUI import CoreData
```

```
struct PreviewProviderHelper { static let sharedContainer: NSPersistentContainer = { let container =
NSPersistentContainer(name: "Database") let description = NSPersistentStoreDescription()
description.type = NSInMemoryStoreType container.persistentStoreDescriptions = [description]
```

```
        container.loadPersistentStores { _, error in
            if let error = error {
                print("Erro ao carregar container de teste: \(error)")
            }
        }
        return container
    }()

    static var sampleBook: Books {
        let context = sharedContainer.viewContext

        let book = Books(context: context)
```

```

        book.bookTitle = "Clean Code"
        book.bookAuthor = "Robert C. Martin"
        book.bookCategory = "Tecnologia"
        book.bookCurrentPage = 45
        book.bookGoal = 200
        book.bookTotalPages = 425
        book.isTimerRunning = false
        book.wasLastPageAdded = false

        return book
    }
}

```

```

}

#Preview { ZStack { HStack(spacing: 16) { BookCardView( book: PreviewProviderHelper.sampleBook ) } }
.environment(.managedObjectContext, PreviewProviderHelper.sharedContainer.viewContext) }

```

Arquivo: \./View/Sheets/SelectBookSheetView.swift\

```

swift // // SelectBookSheetView.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos
on 19/08/26. //

```

```

import SwiftUI import CoreData

```

```

struct SelectBookSheetView: View { @Environment(.dismiss) var dismiss @EnvironmentObject var
booksViewModel: BooksViewModel @EnvironmentObject var userSettingsViewModel:
UserSettingsViewModel @EnvironmentObject var stopwatchViewModel: StopwatchViewModel

```

```

@Binding var selectedBook: Books?

@State private var showingDiscardAlert = false

var body: some View {
    NavigationStack {
        ZStack {
            Color("BackgroundColorViews")
                .ignoresSafeArea()

            ScrollView(showsIndicators: false) {
                VStack(alignment: .leading, spacing: 0) {
                    ForEach(booksViewModel.savedBooks, id: \.objectID) {
book in
                        let isSelected = selectedBook == book

                        Button(action: {
                            selectedBook = book

                            userSettingsViewModel.updateUserSettings(newLastBook: book)
                                userSettingsViewModel.fetchUserSettings()

```

```

        stopwatchViewModel.getTotalPages(book: book)
        stopwatchViewModel.getCurrentPage(book: book)

        dismiss()
    }) {
        VStack(spacing: 14) {
            HStack {
                BookCellView(book: book, isSelected:
isSelected)

                Spacer()
            }
            .padding(.horizontal, 24)

            Rectangle()
                .fill(Color.white.opacity(0.15))
                .frame(height: 1)
                .padding(.horizontal, 24)
        }
        .padding(.top, 14)
        .contentShape(Rectangle())
    }
    .buttonStyle(.plain)
}
}
.padding(.top, 16)
}
}
.navigationBarTitleDisplayMode(.inline)
.toolbar {
    SheetHeaderView(
        title: "Livros salvos",
        actionIcon: "checkmark",
        hasChanges: false,
        showingDiscardAlert: $showingDiscardAlert,
        onCancel: { dismiss() },
        onConfirm: { dismiss() },
        onDiscard: { dismiss() }
    )
}
.onAppear {
    booksViewModel.fetchBooks()
}
.onDisappear{
    userSettingsViewModel.fetchUserSettings()
}
}
}
}

```

```

}

```

```

#Preview { @Previewable @State var selectedBook: Books? = nil

```

```
SelectBookSheetView(selectedBook: $selectedBook)
    .environmentObject(BooksViewModel())
    .environmentObject(UserSettingsViewModel())
    .environmentObject(StopwatchViewModel())
```

```
}
```

Arquivo: \./View/Sheets/NoteSheetView.swift\

```
swift // // NoteSheetView.swift // CH4-Books // // Created by Lucas on 14/08/26. //
```

```
import SwiftUI import PhotosUI import CoreData
```

```
struct NoteSheetView: View {
```

```
    @Environment(\.dismiss) private var dismiss
    @EnvironmentObject private var notesViewModel: NotesViewModel

    let book: Books
    let noteToEdit: Notes?

    @State private var noteTitle: String
    @State private var noteDescription: String
    @State private var noteImages: [SelectableImage]
    @State private var selectedCategory: String

    @State private var initialImagesCount: Int

    @State private var pickedItems: [PhotosPickerItem] = []
    @State private var isShowingPhotoPicker = false
    @State private var isShowingCamera = false

    @State private var showingDiscardAlert = false
    @State private var showErrorAlert = false
    @State private var errorMessage = ""

    private static let maxImages = 3

    private var hasChanges: Bool {
        let initialTitle = noteToEdit?.noteTitle ?? ""
        let initialDescription = noteToEdit?.noteDescription ?? ""
        let initialCategory = noteToEdit?.noteCategory ?? ""

        let titleChanged = noteTitle != initialTitle
        let descriptionChanged = noteDescription != initialDescription
        let categoryChanged = selectedCategory != initialCategory
        let imagesChanged = noteImages.count != initialImagesCount

        return titleChanged || descriptionChanged || categoryChanged ||
        imagesChanged
```

```

}

init(book: Books, noteToEdit: Notes? = nil) {
    self.book = book
    self.noteToEdit = noteToEdit

    _noteTitle = State(initialValue: noteToEdit?.noteTitle ?? "")
    _noteDescription = State(initialValue: noteToEdit?.noteDescription ??
    "")
    _selectedCategory = State(initialValue: noteToEdit?.noteCategory ?? "")

    let existingImages = ((noteToEdit?.notePhoto as? [Data]) ?? [])
        .compactMap { UIImage(data: $0) }
        .prefix(Self.maxImages)
        .map { SelectableImage(image: $0) }

    _noteImages = State(initialValue: Array(existingImages))
    _initialImagesCount = State(initialValue: existingImages.count)
}

private var toolbarTitle: String {
    noteToEdit != nil ? "Editar nota" : "Adicionar nota"
}

var body: some View {
    NavigationStack {
        ZStack {
            Color("BackgroundColorViews")
                .ignoresSafeArea()

            ScrollView(showsIndicators: false) {
                VStack(alignment: .leading, spacing: 17) {
                    noteContentHeader
                    titleField
                    noteTextEditor
                    mediaSection
                }
                .padding(.horizontal)
                .padding(.top, 20)
            }
            .alert("Erro ao executar a ação.", isPresented:
$showErrorAlert) {
                Button("Tentar novamente", role: .cancel) { }
            } message: {
                Text(errorMessage)
            }
        }
        .navigationBarTitleDisplayMode(.inline)
        .toolbar {
            SheetHeaderView(
                title: toolbarTitle,
                actionIcon: "checkmark",
                hasChanges: hasChanges,
                showingDiscardAlert: $showingDiscardAlert,

```

```

        onCancel: {
            hideKeyboard()
            dismiss()
        },
        onConfirm: handleConfirm,
        onDiscard: {
            hideKeyboard()
            dismiss()
        }
    )
}
.onTapGesture {
    #if canImport(Uikit)
        hideKeyboard()
    #endif
}
.scrollDismissesKeyboard(.interactively)

.fullScreenCover(isPresented: $isShowingCamera) {
    CameraPicker { image in
        appendImage(image)
    }
    .ignoresSafeArea()
}
}
}

private var noteContentHeader: some View {
    VStack(alignment: .leading, spacing: 0) {
        Text("Livro: \$(book.bookTitle ?? "Sem título")")
            .font(.system(.title3))
            .foregroundColor(Color("TextFieldPlaceholderColor"))
            .padding(.bottom, 6)
            .padding(.leading, 4)

        Text("Conteúdo da nota")
            .font(.system(.title, weight: .medium))
            .foregroundColor(Color("Texts"))
            .padding(.bottom, 6)
            .padding(.leading, 4)

        TipsComponent(content: "Adicione um título e um conteúdo para a sua
nota.")
    }
}

private var titleField: some View {
    TextFieldSheets(
        text: $noteTitle,
        placeholder: "Adicione o título",
        label: nil
    )
}

```

```

private var noteTextEditor: some View {
    ZStack(alignment: .topLeading) {
        if noteDescription.isEmpty {
            Text("Escreva sua nota...")
                .foregroundColor(Color("TextFieldPlaceholderColor"))
                .padding(.horizontal, 16)
                .padding(.vertical, 16)
                .zIndex(1)
                .allowsHitTesting(false)
                .font(.system(.body, weight: .regular))
        }

        TextEditor(text: $noteDescription)
            .padding(8)
            .scrollContentBackground(.hidden)
            .cornerRadius(10)
            .frame(height: 250)
            .foregroundColor(.white)
            .font(.system(.body, weight: .regular))
            .overlay(
                RoundedRectangle(cornerRadius: 20)
                    .stroke(Color("LinesColor"), lineWidth: 0.5)
            )
    }
}

private var mediaSection: some View {
    VStack(alignment: .leading, spacing: 12) {
        if !noteImages.isEmpty {
            mediaThumbnailsRow
        }
        mediaSourceMenu
        TipsComponent(content: "Você pode adicionar até 3 fotos em uma mesma nota.")
    }
    .photosPicker(
        isPresented: $isShowingPhotoPicker,
        selection: $pickedItems,
        maxSelectionCount: max(1, Self.maxImages - noteImages.count),
        matching: .images,
        photoLibrary: .shared()
    )
    .onChange(of: pickedItems) { _, newItems in
        guard !newItems.isEmpty else { return }
        Task { await loadPickedItems(newItems) }
    }
}

private var mediaThumbnailsRow: some View {
    ScrollView(.horizontal, showsIndicators: false) {
        HStack(spacing: 12) {
            ForEach(noteImages) { item in
                MediaThumbnailView(image: item.image) {
                    noteImages.removeAll { $0.id == item.id }
                }
            }
        }
    }
}

```

```

    }
    }
    }
}

private var mediaSourceMenu: some View {
    Menu {
        Button {
            isShowingCamera = true
        } label: {
            Label("Tirar foto", systemImage: "camera")
        }
        .disabled(!UIImagePickerController.isSourceTypeAvailable(.camera))

        Button {
            isShowingPhotoPicker = true
        } label: {
            Label("Escolher da galeria", systemImage: "photo.on.rectangle")
        }
    } label: {
        HStack {
            Image(systemName: "camera.viewfinder")

            Text(noteImages.isEmpty ? "Adicionar fotos" : "Adicionar mais
fotos (\(noteImages.count)/\((Self.maxImages))")
                .font(.system(.body, weight: .regular))

            Spacer()

            Image(systemName: "chevron.up.chevron.down")
                .font(.footnote.weight(.semibold))
        }
        .foregroundColor(Color("AddNoteImage"))
        .padding()
        .contentShape(Rectangle())
        .overlay(
            RoundedRectangle(cornerRadius: 100)
                .stroke(Color("LinesColor"), lineWidth: 0.5)
        )
    }
    .menuOrder(.fixed)
    .buttonStyle(.plain)
    .disabled(noteImages.count >= Self.maxImages)
    .opacity(noteImages.count >= Self.maxImages ? 0.5 : 1.0)
}

@MainActor
private func loadPickedItems(_ items: [PhotosPickerItem]) async {
    for item in items {
        guard noteImages.count < Self.maxImages else { break }
        guard let data = try? await item.loadTransferable(type: Data.self),
            let image = UIImage(data: data) else { continue }
    }
}

```

```
        noteImages.append(SelectableImage(image: image))
    }
    pickedItems.removeAll()
}

private func appendImage(_ image: UIImage?) {
    guard let image, noteImages.count < Self.maxImages else { return }
    noteImages.append(SelectableImage(image: image))
}

private func handleConfirm() {
    hideKeyboard()

    do {
        if let note = noteToEdit {
            try notesViewModel.updateNote(
                note: note,
                noteTitle: noteTitle,
                noteDescription: noteDescription,
                notePhotos: noteImages.map(\.image)
            )
        } else {
            try notesViewModel.addNote(
                noteTitle: noteTitle,
                noteDescription: noteDescription,
                noteCategory: selectedCategory,
                notePhotos: noteImages.map(\.image),
                to: book
            )
        }

        notesViewModel.fetchNotes(for: book)
        dismiss()

    } catch let error as LocalizedError {
        errorMessage = error.errorDescription ?? "Ocorreu um erro desconhecido."
        showErrorAlert = true
    } catch {
        errorMessage = "Erro inesperado."
        showErrorAlert = true
    }
}

private func hideKeyboard() {
    UIApplication.shared.sendAction(
        #selector(UIResponder.resignFirstResponder),
        to: nil,
        from: nil,
        for: nil
    )
}
```

```
} #Preview { NoteSheetView(book: PreviewProviderHelper.sampleBook)
.environmentObject(NotesViewModel()) }
```

Arquivo: \./View/Sheets/NoteDetailView.swift\

```
swift // // NoteDetailView.swift // CH4-Books // // Created by Lucas on 18/08/26. //
```

```
import SwiftUI
```

```
struct NoteDetailView: View { @EnvironmentObject private var notesViewModel: NotesViewModel
```

```

    @ObservedObject var note: Notes

    @Environment(\.dismiss) private var dismiss

    @State private var isShowingEditSheet = false
    @State private var showErrorAlert = false
    @State private var errorMessage = ""

    @State private var showingDeleteAlert = false

    private var noteImages: [UIImage] {
        if let photosData = note.notePhoto as? [Data] {
            return photosData.compactMap { UIImage(data: $0) }
        }
        return []
    }

    var body: some View {
        NavigationStack {
            ZStack(alignment: .top) {
                Color(.backgroundColorViews)
                    .ignoresSafeArea()

                ScrollView(showsIndicators: false) {
                    VStack(alignment: .leading, spacing: 24) {

                        if !noteImages.isEmpty {
                            ScrollView(.horizontal, showsIndicators: false) {
                                HStack(spacing: 16) {
                                    ForEach(Array(noteImages.enumerated()), id:
\
.offset) { _, image in
                                        Image(uiImage: image)
                                            .resizable()
                                            .scaledToFill()
                                            .frame(width: 240, height: 300)
                                            .cornerRadius(16)
                                            .clipped()
                                    }
                                }
                            .padding(.horizontal, 24)
                        }
                    }
                }
            }
        }
    }

```

```

    }

    VStack(alignment: .leading, spacing: 16) {
        Text(note.noteTitle ?? "Sem título")
            .font(.system(.title, weight: .medium))
            .foregroundColor(.title)
            .multilineTextAlignment(.leading)

        Text(note.noteDescription ?? "Sem descrição")
            .font(.system(.body, weight: .light))
            .foregroundColor(Color(.title))
            .lineSpacing(5)
            .multilineTextAlignment(.leading)
    }
    .padding(.horizontal, 24)
}
.frame(maxWidth: .infinity, alignment: .leading)
.padding()
.padding(.bottom, 40)
}
.alert("Erro ao executar a ação.", isPresented:
$showErrorAlert) {
    Button("Tentar novamente", role: .cancel) { }
} message: {
    Text(errorMessage)
}
.toolbar {
    NoteDetailsToolbar(
        title: "Nota",
        onDelete: {
            showingDeleteAlert.toggle()
        },
        onEdit: {
            isShowingEditSheet.toggle()
        },
    )
}
.alert("Apagar Nota", isPresented: $showingDeleteAlert) {
    Button("Cancelar", role: .cancel) { }
    Button("Apagar", role: .destructive) {
        do{
            try notesViewModel.deleteNote(note: note)
            notesViewModel.fetchNotes(for: note.book)
            dismiss()
        } catch let error as LocalizedError {
            errorMessage = error.errorDescription ?? "Ocorreu
um erro desconhecido."
            showErrorAlert = true
        } catch {
            errorMessage = "Erro inesperado."
            showErrorAlert = true
        }
    }
} message: {

```

}

```

@Binding var readedPages: String

@State private var showCustomDiscardAlert = false

var onDismiss: () -> Void = {}
var onSave: () -> Void = {}

private var hours: Int {
    minutesPerDay / 60
}

private var minutes: Int {
    minutesPerDay % 60
}

var body: some View {
    NavigationStack {
        VStack(spacing: 20) {
            if isPickerShown {
                VStack(spacing: 4) {
                    Text("Editar objetivo diário")
                        .font(.title2.weight(.semibold))
                        .foregroundColor(.white)
                    Text("Defina quantos minutos você\ndeseja ler por dia")
                        .font(.callout)
                        .foregroundColor(.white.opacity(0.7))
                        .multilineTextAlignment(.center)
                }
                VStack(spacing: 0) {
                    Text("Tempo por dia")
                        .font(.callout)
                        .foregroundColor(.white)
                        .padding(.top, 12)
                    HStack(spacing: 0) {
                        Picker("Horas", selection: Binding(
                            get: { hours },
                            set: { newHours in minutesPerDay = (newHours *
60) + minutes }
                        )) {
                            ForEach(0..<24, id: \.self) { hour in
                                Text("\(hour) h").tag(hour)
                            }
                        }
                        .pickerStyle(.wheel)

                        Picker("Minutos", selection: Binding(
                            get: { minutes },
                            set: { newMinutes in minutesPerDay = (hours *
60) + newMinutes }
                        )) {
                            ForEach(0..<60, id: \.self) { minute in
                                Text("\(minute) min").tag(minute)

```

```

        }
    }
    .pickerStyle(.wheel)
}
.frame(height: 120)
.padding(.horizontal, 8)
}
.overlay(
    RoundedRectangle(cornerRadius: 20)
        .stroke(Color("LinesColor"), lineWidth: 0.3)
)
Spacer()
} else {
    VStack(spacing: 4) {
        Text("Adicionar leitura do livro")
            .font(.title2.weight(.semibold))
            .foregroundColor(.white)
        Text("Qual foi a última página lida?")
            .font(.callout)
            .foregroundColor(.white.opacity(0.7))
            .multilineTextAlignment(.center)
    }
    VStack(spacing: 0) {
        TextFieldSheets(text: $readedPages, placeholder:
"Informe a última página lida")
            .keyboardType(.numberPad)
    }
    .padding(.horizontal)
}
}
.navigationBarTitleDisplayMode(.inline)
.padding(.horizontal, 20)
.toolbar {
    if isPickerShown {
        SheetHeaderView(
            title: "",
            actionIcon: "checkmark",
            hasChanges: false,
            showingDiscardAlert: $showAlert,
            onCancel: { onDismiss() },
            onConfirm: { onSave() },
            onDiscard: { onDismiss() }
        )
    } else {
        ToolbarItem(placement: .navigationBarLeading) {
            Button(action: {
                showCustomDiscardAlert = true
            }) {
                Image(systemName: "xmark")
                    .fontWeight(.bold)
                    .foregroundColor(.white)
            }
        }
    }
}

```

```
// .foregroundColor(Color("ActionColor")) // Button("Cancelar") { //
// }
```

```

        }

        ToolbarItem(placement: .navigationBarTrailing) {
            Button(action: {
                onSave()
            }) {
                Image(systemName: "checkmark")
                    .fontWeight(.bold)
                    .foregroundColor(Color("ActionColor"))
            }
            .tint(Color("ActionColor"))
        }
    }
}

.interactiveDismissDisabled(!isPickerShown)

.alert("Atenção", isPresented: $showCustomDiscardAlert) {
    Button("Descartar tempo", role: .destructive) {
        onDismiss()
    }
    Button("Voltar", role: .cancel) { }
} message: {
    Text("Tem certeza que deseja não colocar a última página lida?
Se fizer isso, seu tempo de leitura será desconsiderado e não pode ser
recuperado.")
}
}
}
}

```

```
}
```

```
#Preview("Edit Daily Goal - SheetHeaderView") { struct PreviewWrapper: View { @State private var
isPresented = true @State private var goalMinutes = 15 // exemplo 0h 15min @State private var
tempPages = ""
```

```

var body: some View {
    ZStack {
        Color.black.ignoresSafeArea()

        Button("Abrir Sheet") {
            isPresented = true
        }
        .buttonStyle(.borderedProminent)
    }
    .sheet(isPresented: $isPresented) {
        BottomSheet(
            minutesPerDay: $goalMinutes,

```

```

        isPickerShown: true,
        readedPages: $tempPages,
        onDismiss: { isPresented = false },
        onSave: { isPresented = false }
    )
    .presentationDetents([.height(340)])
    .presentationDragIndicator(.visible)
    .presentationBackground(Color("BackgroundColorViews"))
    }
}

return PreviewWrapper()

```

```

}

```

Arquivo: \./View/Sheets/PageProgressSheet.swift\

```

swift // // PageProgressSheet.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on
20/08/26. //

```

```

import SwiftUI

```

```

struct PageProgressSheetView: View { let bookTitle: String let totalPages: Int

```

```

    @Binding var currentPage: Int
    @State private var tempPage: Int
    @State private var showAlert = false

    var onDismiss: () -> Void = {}
    var onSave: (Int) -> Void = { _ in }

    init(bookTitle: String, totalPages: Int, currentPage: Binding<Int>,
onDismiss: @escaping () -> Void = {}, onSave: @escaping (Int) -> Void = { _
in }) {
        self.bookTitle = bookTitle
        self.totalPages = max(totalPages, 1)
        self._currentPage = currentPage
        self._tempPage = State(initialValue: currentPage.wrappedValue)
        self.onDismiss = onDismiss
        self.onSave = onSave
    }

    var body: some View {
        NavigationStack {
            VStack(spacing: 16) {
                VStack(spacing: 6) {
                    Text("Em qual página você parou no livro")
                        .font(.body)
                        .foregroundColor(.white)
                        .multilineTextAlignment(.center)

```

```

        Text("\ (bookTitle)"?)
            .font(.title3.weight(.semibold))
            .foregroundColor(.white)
            .multilineTextAlignment(.center)
            .lineLimit(2)
    }

    TextFieldSheets(
        text: .constant(""),
        placeholder: "Digite o número da página...",
    )
    .padding(.horizontal)
}
.navigationBarTitleDisplayMode(.inline)
.padding(.horizontal, 20)
.toolbar {
    SheetHeaderView(
        title: "",
        actionIcon: "checkmark",
        hasChanges: false,
        showingDiscardAlert: $showAlert,
        onCancel: { onDismiss() },
        onConfirm: {
            currentPage = tempPage
            onSave(tempPage)
            onDismiss()
        },
        onDiscard: { onDismiss() }
    )
}
}
}
}

```

```

}

```

```

#Preview("Page Progress Sheet") { struct PreviewWrapper: View { @State private var isPresented = true
@State private var page = 42

```

```

var body: some View {
    ZStack {
        Color.black.ignoresSafeArea()

        Button("Abrir Mini Sheet") {
            isPresented = true
        }
        .buttonStyle(.borderedProminent)
    }
    .sheet(isPresented: $isPresented) {
        PageProgressSheetView(
            bookTitle: "Hush, Hush",

```

```

        totalPages: 384,
        currentPage: $page,
        onDismiss: { isPresented = false },
        onSave: { newPage in
            print("Nova página salva: \(newPage)")
        }
    )
    .presentationDetents([.height(220)])
    .presentationDragIndicator(.visible)
    .presentationBackground(Color("BackgroundColorViews"))
}
}
}

return PreviewWrapper()

```

```

}

```

Arquivo: \./View/Sheets/BookSheetView.swift\

swift // // AddBookSheetView.swift // CH4-Books // // Created by Julio Sampaio on 15/08/26. //

import Foundation import SwiftUI import PhotosUI

struct BookSheetView: View { @Environment(.dismiss) var dismiss @EnvironmentObject var photoLibraryViewModel: PhotoLibraryViewModel @EnvironmentObject var booksViewModel: BooksViewModel

```

@State var bookToEdit: Books?
@State private var initialCoverImage: UIImage? = nil

private var hasChanges: Bool {
    if let book = bookToEdit {
        let titleChanged = bookTitle != (book.bookTitle ?? "")
        let authorChanged = bookAuthor != (book.bookAuthor ?? "")
        let categoryChanged = selectedCategory != (book.bookCategory ?? "")
        let pagesChanged = bookTotalPages != (book.bookTotalPages > 0 ?
String(book.bookTotalPages) : "")
        let imageChanged = photoLibraryViewModel.selectedCoverImage !=
initialCoverImage

        return titleChanged || authorChanged || categoryChanged ||
pagesChanged || imageChanged
    } else {
        return !bookTitle.isEmpty
            || !bookAuthor.isEmpty
            || !selectedCategory.isEmpty
            || !bookTotalPages.isEmpty
            || photoLibraryViewModel.selectedCoverImage != nil
    }
}

```

```

}

private var toolbarTitle: String {
    bookToEdit != nil ? "Editar livro" : "Adicionar livro"
}

@State private var showingDiscardAlert: Bool = false
@State private var showErrorAlert = false
@State private var errorMessage = ""

@State private var bookTitle = ""
@State private var selectedCategory: String = ""
@State private var bookTotalPages: String = ""
@State private var bookAuthor: String = ""

var body: some View {
    NavigationStack {
        ZStack {
            Color("BackgroundColorViews")
                .ignoresSafeArea()

            VStack(spacing: 20) {
                ScrollView(showsIndicators: false) {
                    VStack(spacing: 17) {

                        // Capa do livro
                        VStack {
                            PhotosPicker(selection:
$photoLibraryViewModel.selectedItem, matching: .images) {
                                ZStack(alignment: .bottomTrailing) {
                                    Group {
                                        if let selectedImage =
photoLibraryViewModel.selectedCoverImage {
                                            Image(uiImage: selectedImage)
                                                .resizable()
                                                .scaledToFill()
                                                .frame(width: 150, height:
210)

                                            .clipShape(RoundedRectangle(cornerRadius: 16))
                                        } else {
                                            Image("defaultBook")
                                                .resizable()
                                                .scaledToFill()
                                                .frame(width: 172, height:
243)

                                            .background(Color(uiColor:
.systemGray4))
                                        }
                                    }
                                }

                                Image(systemName: "plus")
                            }
                        }
                    }
                }
            }
        }
    }
}

```

```

        .font(.title2)
        .fontWeight(.semibold)
        .foregroundColor(.title)
        .frame(width: 44, height: 44)
        .background(.ultraThinMaterial, in:
Circle())

        .clipShape(Circle())
        .overlay(
            Circle()

.stroke(Color.white.opacity(0.2), lineWidth: 1)
        )
        .padding(.horizontal, 6)
        .padding(.vertical, 6)
    }
}
.buttonStyle(.plain)
.padding(.bottom, 12)
.padding(.trailing, 12)
}

if photoLibraryViewModel.selectedCoverImage != nil
{
    Button(action: {
        withAnimation {

photoLibraryViewModel.selectedCoverImage = nil
                photoLibraryViewModel.selectedItem =
nil

        }
    }) {
        Text("Remover capa")
        .font(.body)
        .foregroundColor(.red)
    }
}

// Detalhes do livro
TextFieldSheets(text: $bookTitle, placeholder:
"Adicione o título", label: "Título")
TextFieldSheets(text: $bookAuthor, placeholder:
"Adicione o autor(a)", label: "Autor(a)")

MenuSheetPicker(
    title: "Categoria",
    placeholder: "Adicione a categoria",
    selectedValue: $selectedCategory,
    options: booksViewModel.bookCategories
)

TextFieldSheets(text: $bookTotalPages, placeholder:
"Adicione a quantidade de páginas", label: "Páginas")
        .keyboardType(.numberPad)
    }
}

```

```

        .padding(.horizontal)
    }
    .padding(.top, 20)
    .navigationBarTitleDisplayMode(.inline)
    .toolbar {
        SheetHeaderView(
            title: toolbarTitle,
            actionIcon: "checkmark",
            hasChanges: hasChanges,
            showingDiscardAlert: $showingDiscardAlert,
            onCancel: { dismiss() },
            onConfirm: {
                handleSave()
            },
            onDiscard: { dismiss() }
        )
    }
    .alert("Erro ao executar a ação.", isPresented:
$showErrorAlert) {
        Button("Tentar novamente", role: .cancel) { }
    } message: {
        Text(errorMessage)
    }
    .onAppear {
        if let book = bookToEdit {
            bookTitle = book.bookTitle ?? ""
            bookAuthor = book.bookAuthor ?? ""
            selectedCategory = book.bookCategory ?? ""
            bookTotalPages = book.bookTotalPages > 0 ?
String(book.bookTotalPages) : ""

            if let imageData = book.bookCover, let image =
UIImage(data: imageData) {
                photoLibraryViewModel.selectedCoverImage =
image

                initialCoverImage = image
            } else {
                photoLibraryViewModel.selectedCoverImage = nil
                initialCoverImage = nil
            }
        } else {
            photoLibraryViewModel.selectedCoverImage = nil
            initialCoverImage = nil
        }
    }

    .onTapGesture {
        #if canImport(Uikit)
            hideKeyboard()
        #endif
    }
    .scrollDismissesKeyboard(.interactively)
}
}

```

```
    }
}
private func handleSave() {
    if let book = bookToEdit {
        do {
            try booksViewModel.updateBook(
                book: book,
                bookTitle: bookTitle,
                bookAuthor: bookAuthor,
                bookCover: photoLibraryViewModel.selectedCoverImage,
                bookCategory: selectedCategory,
                bookTotalPages: bookTotalPages
            )
            booksViewModel.fetchBooks()
            dismiss()
        } catch let error as LocalizedError {
            errorMessage = error.errorDescription ?? "Ocorreu um erro
desconhecido."
            showErrorAlert = true
        } catch {
            errorMessage = "Erro inesperado."
            showErrorAlert = true
        }
    } else {
        do {
            try booksViewModel.addBook(
                bookTitle: bookTitle,
                bookAuthor: bookAuthor,
                bookCover: photoLibraryViewModel.selectedCoverImage,
                bookCategory: selectedCategory,
                bookTotalPages: bookTotalPages
            )
            booksViewModel.fetchBooks()
            dismiss()
        } catch let error as LocalizedError {
            errorMessage = error.errorDescription ?? "Ocorreu um erro
desconhecido."
            showErrorAlert = true
        } catch {
            errorMessage = "Erro inesperado."
            showErrorAlert = true
        }
    }
}
```

```
} #Preview { BookSheetView(bookToEdit: nil) .environmentObject(PhotoLibraryViewModel())
.environmentObject(BooksViewModel()) }
```

```
swift // // ContentView.swift // CH4-Books // // Created by Julio Sampaio on 13/08/26. //
```

```
import SwiftUI import CoreData
```

```
struct ContentView: View { @State private var searchText = "" @State private var selectedBook: Books? = nil @Namespace private var stopwatchNamespace
```

```

@EnvironmentObject var booksViewModel: BooksViewModel

init() {
    let appearance = UITabBarAppearance()

    // mantém o desfoque/transparência original do sistema
    appearance.configureWithDefaultBackground()

    let actionColor = UIColor(named: "ActionColor") ?? UIColor.orange

    // configura a cor quando o ícone ESTÁ selecionado
    appearance.stackedLayoutAppearance.selected.iconColor = actionColor
    appearance.stackedLayoutAppearance.selected.titleTextAttributes =
    [.foregroundColor: actionColor]

    // configura a cor quando o ícone NÃO ESTÁ selecionado (Cinza padrão)
    appearance.stackedLayoutAppearance.normal.iconColor =
    UIColor.systemGray
    appearance.stackedLayoutAppearance.normal.titleTextAttributes =
    [.foregroundColor: UIColor.systemGray]

    // aplica as regras na TabBar do app
    UITabBar.appearance().standardAppearance = appearance
    UITabBar.appearance().scrollEdgeAppearance = appearance
}

var body: some View {

    TabView {
        Tab("Estante", systemImage: "book"){
            BookCaseView()

```

```

        // .tint(.blue) // .environmentObject(PhotoLibraryViewModel()) } Tab("Cronômetro", systemImage: "timer")
        { StopwatchInitialView( namespace: stopwatchNamespace, selectedBook: selectedBook ) // .tint(.blue) }
        Tab(role: .search){ BookSearchView() // .tint(.blue) }

```

```
    }
```

```
// .tint(Color("ActionColor")) } }
```

```

#Preview { ContentView() .environmentObject(UserSettingsViewModel())
.environmentObject(BooksViewModel()) .environmentObject(StopwatchViewModel())

```

```
.environmentObject(PhotoLibraryViewModel()) .environmentObject(NotesViewModel()) }
```

Arquivo: \./View/Books/BookSearchView.swift\

```
swift // // BookSearchView.swift // CH4-Books // // Created by Lucas on 16/08/26. //
```

```
import SwiftUI import CoreData
```

```
struct BookSearchView: View { @EnvironmentObject var booksViewModel: BooksViewModel
@EnvironmentObject var notesViewModel: NotesViewModel @State private var searchText = "" @State
private var isSearchPresented = false @State private var selectedBook: Books? @State private var
selectedNote: Notes?
```

```
var filteredBooks: [Books] {
    if searchText.isEmpty {
        return booksViewModel.savedBooks
    } else {
        return booksViewModel.savedBooks.filter { book in
            book.bookTitle?.localizedCaseInsensitiveContains(searchText) ??
false
        }
    }
}

var filteredNotes: [Notes] {
    if searchText.isEmpty {
        return notesViewModel.savedNotes
    } else {
        return notesViewModel.savedNotes.filter { note in
            note.noteTitle?.localizedCaseInsensitiveContains(searchText) ??
false
        }
    }
}

var body: some View {
    NavigationStack {
        ZStack {
            Color("BackgroundColorViews")
                .ignoresSafeArea()

            List {
                if !filteredBooks.isEmpty {
                    Section {
                        ForEach(filteredBooks, id: \.self) { book in
                            Button {
                                selectedBook = book
                            } label: {
                                BookCellView(book: book)
                            }
                            .buttonStyle(.plain)
                        }
                    }
                }
            }
        }
    }
}
```

```

        .listRowBackground(Color("CardNoteColor"))
    }
    } header: {
        Text("Livros")
        .font(.bitter(.medium, style: .title))
        .fontWeight(.semibold)
        .foregroundStyle(Color("Texts"))
    }
    .foregroundStyle(.white)
}

if !filteredNotes.isEmpty {
    Section {
        ForEach(filteredNotes, id: \.self) { note in
            Button {
                selectedNote = note
            } label: {
                NoteCellView(note: note, noteViewModel:
notesViewModel)
            }
            .buttonStyle(.plain)
            .listRowBackground(Color("CardNoteColor"))
        }
    } header: {
        Text("Notas")
        .font(.bitter(.medium, style: .title))
        .fontWeight(.semibold)
        .foregroundStyle(Color("Texts"))
    }
    .foregroundStyle(.white)
}
}
.scrollContentBackground(.hidden)
.contentMargins(.top, 0, for: .scrollContent)

.overlay {
    if filteredBooks.isEmpty && filteredNotes.isEmpty {
        ContentUnavailableView {
            Text("Nenhum resultado encontrado")
            .font(.title3)
            .fontWeight(.semibold)
            .foregroundStyle(Color("Texts"))
        } description: {
            Text("Tente buscar por outro título de livro ou
nota")
            .font(.subheadline)
            .foregroundStyle(.white.opacity(0.7))
        }
    }
}
}
.navigationTitle("Buscar")
.navigationBarTitleDisplayMode(.inline)
.searchable(

```

```

        text: $searchText,
        isPresented: $isSearchPresented,
        prompt: "Buscar livros ou notas"
    )
    .onAppear {
        isSearchPresented = true
        booksViewModel.fetchBooks()
        notesViewModel.fetchNotes()
    }
    .sheet(item: $selectedBook, onDismiss: {
        booksViewModel.fetchBooks()
        notesViewModel.fetchNotes()
    }) { book in
        BookDetailView(bookViewModel: booksViewModel, book: book)
    }
    .sheet(item: $selectedNote, onDismiss: {
        booksViewModel.fetchBooks()
        notesViewModel.fetchNotes()
    }) { note in
        NoteDetailSheetView(note: note)
    }
    .onTapGesture {
        #if canImport(UiKit)
            hideKeyboard()
        #endif
    }
    .scrollDismissesKeyboard(.interactively)
}

}
}

```

```

}

```

```

#Preview { let context = CoreDataManager.shared.viewContext

```

```

let book1 = Books(context: context)
book1.bookTitle = "Dom Casmurro"
book1.bookAuthor = "Machado de Assis"
book1.bookCategory = "Romance"
book1.bookTotalPages = 256
book1.bookCurrentPage = 120
book1.isTimerRunning = false
book1.wasLastPageAdded = true

let book2 = Books(context: context)
book2.bookTitle = "O Pequeno Príncipe"
book2.bookAuthor = "Antoine de Saint-Exupéry"
book2.bookCategory = "Infantil"
book2.bookTotalPages = 96
book2.bookCurrentPage = 96
book2.isTimerRunning = false

```

```

book2.wasLastPageAdded = true

let note1 = Notes(context: context)
note1.noteTitle = "Anotações sobre "
note1.noteDescription = "Reflexão sobre a personagem e sua ambiguidade."
note1.noteCategory = "Pensamento"
note1.book = book1

let note2 = Notes(context: context)
note2.noteTitle = "Ideias para resumo"
note2.noteDescription = "Pontos principais do capítulo 3."
note2.noteCategory = "Resumo"
note2.book = book2

try? context.save()

let booksVM = BooksViewModel()
let notesVM = NotesViewModel()

return BookSearchView()
    .environmentObject(booksVM)
    .environmentObject(notesVM)
}

```

Arquivo: \./View/Books/BookDetailsView.swift\

swift // // BookDetailView.swift // CH4-Books // // Created by Lucas on 17/08/26. //

import SwiftUI import CoreData

```

struct BookDetailView: View { @ObservedObject var bookViewModel: BooksViewModel
@ObservedObject var userSettingsViewModel = UserSettingsViewModel() @StateObject private var
notesViewModel = NotesViewModel() @State private var isPresentedAddNote: Bool = false @State
private var isPresentedEditBook: Bool = false @State private var isPresentedBottomSheet: Bool = false
@State private var tempGoalMinutes: Int = 15

```

```

@State private var tempReadedPages: String = ""

@EnvironmentObject var photoLibraryViewModel: PhotoLibraryViewModel

@State private var showErrorAlert = false
@State private var errorMessage = ""

var book: Books

//apagar assim que possivel

```

```
// private var currentBook: Books { // book ?? bookViewModel.savedBooks.first // }
```

```

var body: some View {
    NavigationStack{
        ZStack {
            Color(.orange)
                .opacity(0.1)
                .ignoresSafeArea()

            ScrollView(showsIndicators: false) {
                VStack() {

                    BookInstanceDetailView(book: book)
                        .environmentObject(PhotoLibraryViewModel())
                        .id(book.bookCover ?? Data())

                    CardTotalPages(totalPages:
bookViewModel.countBookReadedPages(book: book))
                        .padding(.horizontal)

                    NotesHeaderview(isPresentedAddNote:
$isPresentedAddNote)
                        .padding(.horizontal, 24)
                        .padding(.top, 10)

                    VStack(spacing: 16) {

                        VStack{
                            VStack() {
                                NotesSectionView(notes:
Array(notesViewModel.savedNotes.prefix(3))) {
                                    notesViewModel.fetchNotes(for: book)
                                }
                                Divider()
                                    .frame(height: 0.3)
                                    .background(Color("LinesColor"))
                                    .padding(.horizontal, 28)
                            }
                        }

                        NavigationLink(destination: MyNotesListView(book:
book)) {

                            Text(notesViewModel.savedNotes.isEmpty ? "Ver
anotações" : "Ver mais anotações...")
                                .font(.system(size: 14, weight: .semibold))
                                .foregroundColor(Color(.action))
                                .frame(maxWidth: .infinity, alignment:
.leading)
                                .padding(.horizontal,28)

```

```

                .padding(.vertical)
                .padding(.bottom, 10)
            }

        }
        .background(Color("CardNoteColor"))
        .cornerRadius(30)
        .padding(.horizontal)
        VStack(spacing: 16) {

            ButtonAction(text: "Adicionar leitura", isGlass:
true) {
                isPresentedBottomSheet.toggle()
            }
            .buttonStyle(.glass)
            .padding(.horizontal, 24)
            .padding(.top, 16)

            // Button preenchido com cor
            ButtonAction(text: "Iniciar leitura", colorButton:
"ActionColor")
                .padding(.horizontal, 24)
        }
        .padding(.bottom, 40)
    }
}
.onAppear {
    notesViewModel.fetchNotes(for: book)
}
.onDisappear {
    bookViewModel.fetchBooks()
}
.sheet(isPresented: $isPresentedAddNote, onDismiss: {
    notesViewModel.fetchNotes(for: book)
}) {
    NoteSheetView(book: book)
        .environmentObject(notesViewModel)
}

.sheet(isPresented: $isPresentedBottomSheet) {
    BottomSheet(
        minutesPerDay: $tempGoalMinutes,
        isPickerShown: false,
        readedPages: $tempReadedPages,
        onDismiss: {
            isPresentedBottomSheet = false
        },
        onSave: {

            do{
                try bookViewModel.updateCurrentPage(book: book,
currentPage: tempReadedPages)
            } catch let error as LocalizedError {

```

```

        errorMessage = error.errorDescription ?? "Ocorreu
um erro desconhecido."
        showErrorAlert = true
    } catch {
        errorMessage = "Erro inesperado."
        showErrorAlert = true
    }

    isPresentedBottomSheet = false
}
)
.onAppear {
    tempGoalMinutes = userSettingsViewModel.dailyGoal
    tempReadedPages = ""
}
.presentationDetents([.height(340)])
.presentationDragIndicator(.visible)
.presentationBackground(Color("BackgroundColorViews"))
}
.sheet(isPresented: $isPresentedEditBook, onDismiss: {
    bookViewModel.fetchBooks()
}){
    BookSheetView(bookToEdit: book)
        .environmentObject(photoLibraryViewModel)
        .environmentObject(bookViewModel)
}
.alert("Erro ao executar a ação.", isPresented: $showErrorAlert) {
    Button("Tentar novamente", role: .cancel) { }
} message: {
    Text(errorMessage)
}

.toolbar{
    BooksDetailsToolbar(onEdit: {
        isPresentedEditBook.toggle()
    })
}
}
}
}

```

```

} // #Preview { // BookDetailView(bookViewModel: BooksViewModel()) //
.environmentObject(PhotoLibraryViewModel()) // .environmentObject(NotesViewModel()) //}

```

Arquivo: \./View/Books/BookCaseView.swift\

swift // // BookCaseView.swift (REFATORADO) // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on 15/08/26. //

import SwiftUI

```

struct BookCaseView: View { @EnvironmentObject var bookViewModel: BooksViewModel
@EnvironmentObject var userSettingsViewModel: UserSettingsViewModel @EnvironmentObject var
stopwatchViewModel: StopwatchViewModel @EnvironmentObject var filterViewModel:
BookFilterViewModel

```

```

enum ActiveSheet: Identifiable {
    case addBook
    case editGoal

    var id: Self { self }
}

@State private var activeSheet: ActiveSheet? = nil

@State private var tempGoalMinutes: Int = 15
@State private var tempReadedPages: String = ""

var filteredBooks: [Books] {
    return filterViewModel.applyFilters(to: bookViewModel.savedBooks)
}

let columns = [
    GridItem(.flexible(), spacing: 12),
    GridItem(.flexible(), spacing: 12)
]

var body: some View {
    NavigationStack {
        ZStack {
            Color("BackgroundColorViews")
                .ignoresSafeArea()

            ScrollView {
                VStack (alignment: .leading, spacing: 24) {
                    HStack {
                        TitleComponent(title: "Meus Livros")
                    }
                    .padding(.top, 28)

                    CardTotalPages(totalPages:
bookViewModel.countGeralReadedPages())

                    DailyGoalCardView(
                        onEditAction: {
                            activeSheet = .editGoal
                        }
                    )
                    LazyVGrid(columns: columns, spacing: 12) {
                        ForEach(filteredBooks) { book in
                            NavigationLink(destination:
BookDetailView(bookViewModel: bookViewModel, book: book)){
                                BookCardView(book: book)

```

```

        }
        .buttonStyle(.plain)
    }
}
}
.environmentObject(bookViewModel)
}
.padding(.horizontal, 24)
}
}
.toolbar {
    BookCaseToolbar(onAddClick: {
        activeSheet = .addBook
    }, filterViewModel: filterViewModel)
}
}
.onAppear {
    withAnimation {
        bookViewModel.fetchBooks()
        userSettingsViewModel.fetchUserSettings()
    }

    NotificationManager.shared.requestPermission()
}

.sheet(item: $activeSheet, onDismiss: {
    withAnimation {
        bookViewModel.fetchBooks()
        userSettingsViewModel.fetchUserSettings()
    }
}) { sheet in
    switch sheet {
    case .addBook:
        BookSheetView(bookToEdit: nil)
            .environmentObject(PhotoLibraryViewModel())
            .environmentObject(bookViewModel)

    case .editGoal:
        BottomSheet(
            minutesPerDay: $tempGoalMinutes,
            isPickerShown: true,
            readedPages: .constant(""),
            onDismiss: {
                userSettingsViewModel.fetchUserSettings()
                activeSheet = nil
            },
            onSave: {
                userSettingsViewModel.saveDailyGoal(minutes:
tempGoalMinutes)
                activeSheet = nil
            }
        )
        .onAppear {
            tempGoalMinutes = userSettingsViewModel.dailyGoal
        }
    }
}

```

```

        .presentationDetents([.height(340)])
        .presentationDragIndicator(.visible)
        .presentationBackground(Color("BackgroundColorViews"))
    }
}

.sheet(isPresented: $stopwatchViewModel.showProgressSheet) {
    if let currentBook = userSettingsViewModel.lastBookReaded {
        BottomSheet(
            minutesPerDay: .constant(0),
            isPickerShown: false,
            readedPages: $tempReadedPages,
            onDismiss: {
                stopwatchViewModel.showProgressSheet = false
            },
            onSave: {
                if let newPage = Int16(tempReadedPages) {
                    currentBook.bookCurrentPage = newPage

                    do {
                        try bookViewModel.saveBook()
                    } catch {
                        print("Error when trying to save last readed
page by home: \(error)")
                    }
                }

                let rawMinutes = Int(stopwatchViewModel.totalTime / 60)
                let minutesRead = max(1, rawMinutes)

                userSettingsViewModel.addCompletedReadingTime(minutes:
minutesRead)

                bookViewModel.fetchBooks()
                userSettingsViewModel.fetchUserSettings()
                tempReadedPages = ""
                stopwatchViewModel.showProgressSheet = false
            }
        )
        .presentationDetents([.height(340)])
        .presentationDragIndicator(.visible)
        .presentationBackground(Color("BackgroundColorViews"))
    }
}
}

```

```

} #Preview { BookCaseView() .environmentObject(UserSettingsViewModel())
.environmentObject(PhotoLibraryViewModel()) .environmentObject(BooksViewModel())
.environmentObject(NotesViewModel()) .environmentObject(StopwatchViewModel())
.environmentObject(BookFilterViewModel()) }

```

Arquivo: \./View/Components/StopwatchComponents/TimePickerView.swift\

```

swift //// TimePickerView.swift //// CH4-Books //// Created by Julio Sampaio on 22/08/26. ////
//import Foundation //import SwiftUI // //struct TimePickerView: View { // var body: some View { // // Picker
de Horas // Picker("Horas", selection: Binding( // get: { Int(selectedDuration) / 3600 }, // set: { newHours in
// let currentMinutes = (Int(selectedDuration) % 3600) / 60 //
// // Separado para o compilador não chorar // let hoursInSeconds = Double(newHours * 3600) // let
minutesInSeconds = Double(currentMinutes * 60) // let newTotal = hoursInSeconds + minutesInSeconds
//
// selectedDuration = newTotal // stopwatchViewModel.totalTime = newTotal //
stopwatchViewModel.elapsedTime = newTotal // } // )) { // ForEach(0..<24, id: .self) { hour in // Text("(hour
h)").tag(hour) // } // } // .pickerStyle(.wheel) // // // Picker de Minutos // Picker("Minutos", selection: Binding(
// get: { (Int(selectedDuration) % 3600) / 60 }, // set: { newMinutes in // let currentHours =
Int(selectedDuration) / 3600 //
// // Separado para o compilador não chorar // let hoursInSeconds = Double(currentHours * 3600) // let
minutesInSeconds = Double(newMinutes * 60) // let newTotal = hoursInSeconds + minutesInSeconds //
// selectedDuration = newTotal // stopwatchViewModel.totalTime = newTotal //
stopwatchViewModel.elapsedTime = newTotal // } // )) { // ForEach(0..<60, id: .self) { minute in // Text("
(minute) min").tag(minute) // } // } // .pickerStyle(.wheel) // // } // }

```

Arquivo: \./View/Components/StopwatchComponents/ConcentricRingsView.swift\

```

swift // // ConcentricRingsView.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on
16/08/26. //

```

```

import SwiftUI

```

```

struct ConcentricRingsView: View { var progress: Double private let totalRings = 5

```

```

var body: some View {
    GeometryReader { geometry in
        let baseWidth = geometry.size.width * 0.60
        let baseHeight = geometry.size.height * 0.40

        ZStack {
            ForEach(0..

```

```

        colors: [
            Color("StopWatchColor1"),
            Color("StopWatchColor2")
        ],
        startPoint: .topLeading,
        endPoint: .bottomTrailing
    ),
    lineWidth: index == 0 ? 3.5 : 2.0
)
.opacity(ringOpacity)
.frame(
    width: baseWidth + (step * 2),
    height: baseHeight + (step * 2)
)
.animation(.easeInOut(duration: 0.4), value: progress)
    }
}
.frame(width: geometry.size.width, height: geometry.size.height)
}
}

private func isRingActive(for index: Int) -> Bool {
    if index == 0 { return true }
    let stepProgress = 1.0 / Double(totalRings - 1)
    let threshold = Double(index) * stepProgress
    return progress >= threshold
}

```

```

}

```

Arquivo: \./View/Components/OnboardingComponents/MenuSheetPickerOnboarding.swift\

swift // // MenuSheetPickerOnboarding.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on 20/08/26. //

```

import SwiftUI

```

```

struct MenuSheetPickerOnboarding: View { var title: String? = nil var placeholder: String @Binding var
selectedValue: String var options: [String] var formatOption: (String) -> String = { $0 }

```

```

var body: some View {
    VStack(alignment: .leading, spacing: 6) {
        if let title = title, !title.isEmpty {
            Text(title)
                .font(.system(.title3, weight: .medium))
                .foregroundColor(Color("LinesColor"))
                .padding(.leading, 4)
        }

        Menu {

```

```

        ForEach(options, id: \.self) { option in
            Button(formatOption(option)) {
                selectedValue = option
            }
        }
    } label: {
        HStack {
            Text(selectedValue.isEmpty ? placeholder :
formatOption(selectedValue))
                .font(.system(.body, weight: .regular))
                .foregroundColor(selectedValue.isEmpty ?
Color("TextFieldPlaceholderColor") : .white)

            Spacer()

            Image(systemName: "chevron.up.chevron.down")
                .font(.footnote.weight(.semibold))
                .foregroundColor(Color("ActionColor"))
        }
        .padding()
        .background(
            Capsule()
                .fill(Color("StopwatchSelectors").opacity(0.20))
        )
        .contentShape(Rectangle())
        .overlay(
            RoundedRectangle(cornerRadius: 100)
                .stroke(Color("LinesColor"), lineWidth: 0.5)
        )
    }
    .buttonStyle(.plain)
}
}

```

```

}

```

Arquivo: \./View/Components/NotesComponents/NoteCardView.swift\

swift // // NoteCardView.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on 18/08/26. //

import SwiftUI

struct NotesCardView: View { let imageName: String let tagText: String let title: String let description: String

```

var body: some View {
    HStack(spacing: 16) {
        Image(imageName)

```

```

        .resizable()
        .scaledToFill()
        .frame(width: 90, height: 120)
        .cornerRadius(16)
        .clipped()

VStack(alignment: .leading, spacing: 8) {

    Text(title)
        .font(.title2)
        .fontWeight(.semibold)
        .foregroundColor(.white)

    Text(description)
        .font(.subheadline)
        .foregroundColor(Color("Texts").opacity(0.6))
        .lineLimit(2)
}

Spacer()

Button(action: {
    //acao
}) {
    Image(systemName: "chevron.forward")
        .fontWeight(.semibold)
        .foregroundColor(.primary)
}
.buttonStyle(.plain)
.buttonBorderShape(.circle)
.padding(.trailing, 6)
}
.padding(16)
.background(Color("CardNoteColor"))
.cornerRadius(20)
}

```

```

}

#Preview { NotesCardView( imageName: "defaultBook", tagText: "Referência", title: "Título da nota",
description: "Descrição inicial da primeira..." ) }

```

Arquivo: \./View/Components/NotesComponents/NoteDetailsToolbar.swift\

```
swift // // NoteDetailsToolbar.swift // CH4-Books // // Created by Julio Sampaio on 20/08/26. //
```

```
import Foundation import SwiftUI
```

```
struct NoteDetailsToolbar: ToolbarContent { var title: String var onDelete: () -> Void var onEdit: () -> Void
// var onDeleteTapped: () -> Void
```

```

var body: some ToolbarContent {
    ToolbarItem(placement: .principal) {
        Text(title)
            .foregroundColor(Color("LinesColor"))
            .font(.bitter(.semibold, style: .title2))
    }

    ToolbarItem(placement: .topBarTrailing) {
        Button(action: {
            onDelete()
        }) {
            Image(systemName: "trash")
                .fontWeight(.semibold)
                .foregroundColor(.white)
        }
        .buttonStyle(.borderedProminent)
        .buttonBorderShape(.circle)
        .tint(Color(.red))
    }

    ToolbarItem(placement: .topBarTrailing) {
        Button(action: {
            onEdit()
        }) {
            Image(systemName: "pencil")
                .fontWeight(.semibold)
                .foregroundColor(.primary)
        }
        .buttonStyle(.borderedProminent)
        .buttonBorderShape(.circle)
        .tint(Color("ActionColor"))
    }
}

```

```

}

```

Arquivo: \./View/Components/NotesComponents/NoteCellView.swift\

swift // // NoteCellView.swift // CH4-Books // // Created by Lucas on 18/08/26. // import SwiftUI

struct NoteCellView: View { var note: Notes

```

@ObservedObject var noteViewModel: NotesViewModel

@State private var isShowingDeleteAlert = false
@State private var isEditingSheetPresented = false

@State private var showErrorAlert = false
@State private var errorMessage = ""

```

```

var body: some View {
    VStack(alignment: .leading, spacing: 6) {

        Text(note.noteTitle ?? "Nota sem título")
            .font(.title2)
            .fontWeight(.semibold)
            .foregroundColor(.white)
            .lineLimit(1)

        if let description = note.noteDescription, !description.isEmpty {
            Text(description)
                .font(.body)
                .foregroundColor(.secondary)
                .lineLimit(1)
        }
    }
    .contentShape(Rectangle())
    .contextMenu {
        Button() {

```

```

// isEditingSheetPresented = true } label: { Label("Editar Nota", systemImage: "pencil")
.foregroundColor(.blue) .font(.body) }

```

```

        Divider()

        Button(role: .destructive) {
            isShowingDeleteAlert = true
        } label: {
            Label("Apagar Nota", systemImage: "trash")
                .font(.body)
        }
    }

    .alert("Apagar Nota", isPresented: $isShowingDeleteAlert) {
        Button("Cancelar", role: .cancel) { }
        Button("Apagar", role: .destructive) {
            do{
                try noteViewModel.deleteNote(note: note)

            } catch let error as LocalizedError {
                errorMessage = error.errorDescription ?? "Ocorreu um erro
desconhecido."
                showErrorAlert = true
            } catch {
                errorMessage = "Erro inesperado."
                showErrorAlert = true
            }
        }
    }
    } message: {

```

```
        Text("Tem certeza que deseja apagar esta nota? Essa ação não pode
ser desfeita.")
    }

    .frame(width: 170)
    .cornerRadius(12)
    .padding(.horizontal, 8)
    .padding(.bottom, 4)
}

}
```

Arquivo: \./View/Components/NotesComponents/MyNotesToolBar.swift\

swift // // MyNotesTollBar.swift // CH4-Books // // Created by Lucas on 19/08/26. //

import SwiftUI

struct MyNotesToolBar: ToolbarContent { var onBackClick: () -> Void var onAddClick: () -> Void

```
var body: some ToolbarContent {

    ToolbarItem(placement: .topBarLeading) {
        Button(action: {
            onBackClick()
        }) {
            Image(systemName: "chevron.left")
                .fontWeight(.semibold)
                .foregroundColor(.white)
        }
        .buttonStyle(.plain)
        .buttonBorderShape(.circle)
        .tint(Color.white.opacity(0.2))
    }

    ToolbarItem(placement: .topBarTrailing) {
        Button(action: {
            onAddClick()
        }) {
            Image(systemName: "plus")
                .fontWeight(.semibold)
                .foregroundColor(.primary)
        }
        .buttonStyle(.borderedProminent)
        .buttonBorderShape(.circle)
        .tint(Color("ActionColor"))
    }
}
```

```
// ToolbarItem(placement: .topBarTrailing) { // Button(action: { // onAddClick() // }) { // Image(systemName:
"plus") // .fontWeight(.semibold) // .foregroundColor(.primary) // } // .tint(Color("ActionColor")) //
.buttonStyle(.glass) // .buttonBorderShape(.circle) // } }
```

Arquivo: \./View/Components/BooksComponents/BookCellView.swift\

```
swift // // BookCellView.swift // CH4-Books // // Created by Lucas on 18/08/26. //
```

```
import SwiftUI
```

```
struct BookCellView: View { let book: Books var isSelected: Bool = false
```

```
var progress: Double {
    guard book.bookTotalPages > 0 else { return 0 }
    return min(1, Double(book.bookCurrentPage) /
Double(book.bookTotalPages))
}

var body: some View {
    HStack(spacing: 16) {
        Group {
            if let coverData = book.bookCover, let uiImage = UIImage(data:
coverData) {
                Image(uiImage: uiImage)
                    .resizable()
                    .scaledToFill()
            } else {
                Image("defaultBook")
                    .resizable()
                    .scaledToFill()
                    .background(Color(uiColor: .systemGray5))
            }
        }
        .frame(width: 55, height: 80)
        .clipShape(RoundedRectangle(cornerRadius: 8, style: .continuous))

        VStack(alignment: .leading, spacing: 4) {
            Text(book.bookTitle ?? "Sem Título")
                .font(.title2)
                .fontWeight(.semibold)
                // ✨ Se estiver selecionado, usa a ActionColor, senão usa
branco
                .foregroundColor(isSelected ? Color("ActionColor") :
.white)
                .lineLimit(1)

            Text(book.bookAuthor ?? "Desconhecido")
                .font(.callout)
                .foregroundColor(.white.opacity(0.7))
        }
    }
}
```

```

        Spacer(minLength: 0)

        GeometryReader { geometry in
            ZStack(alignment: .leading) {
                Capsule()
                    .fill(Color.white.opacity(0.15))
                    .frame(height: 6)

                Capsule()
                    .fill(
                        LinearGradient(
                            colors: [Color.yellow, Color.orange],
                            startPoint: .leading,
                            endPoint: .trailing
                        )
                    )
                    .frame(width: geometry.size.width *
CGFloat(progress), height: 6)
            }
        }
        .frame(height: 8)
    }
    .padding(.horizontal, 8)
    .padding(.bottom, 4)
}
}

```

```

}

```

Arquivo: \./View/Components/BooksComponents/TitleComponent.swift\

swift // // TitleComponent.swift // CH4-Books // // Created by Julio Sampaio on 18/08/26. //

import Foundation import SwiftUI

struct TitleComponent: View { var title: String

```

    var body: some View {
        Text(title)
            .font(.bitter(.medium, style: .largeTitle))
            .foregroundColor(Color("TitleColor"))
    }
}

```

```

}

```

#Preview { TitleComponent(title: "Teste") }

Arquivo: \./View/Components/BooksComponents/BookCardView.swift\

```
swift /// BookCardView.swift // CH4-Books /// Created by Julio Sampaio on 17/08/26. //
```

```
import SwiftUI import CoreData
```

```
struct BookCardView: View { @ObservedObject var book: Books
```

```

    @State private var isEditingSheetPresented = false
    @State private var isShowingDeleteAlert = false

    @EnvironmentObject var photoLibraryViewModel: PhotoLibraryViewModel
    @EnvironmentObject var bookViewModel: BooksViewModel

    private var percentageRead: Int {
        guard book.bookTotalPages > 0 else { return 0 }
        let percentage = (Double(book.bookCurrentPage) /
Double(book.bookTotalPages)) * 100.0
        return min(max(Int(percentage), 0), 100)
    }

    var body: some View {
        ZStack(alignment: .bottom) {

            if let uiImage = photoLibraryViewModel.getCoverImage(for: book) {
                Image(uiImage: uiImage)
                    .resizable()
                    .scaledToFill()
                    .frame(width: 170, height: 240)
                    .clipped()
            } else {
                Image("defaultBook")
                    .resizable()
                    .scaledToFill()
                    .frame(width: 170, height: 240)
                    .clipped()
            }

            VStack(alignment: .leading, spacing: 6) {
                Text(book.bookTitle ?? "Sem título")
                    .font(.system(.subheadline, weight: .bold))
                    .foregroundColor(.white)
                    .lineLimit(2)
                    .multilineTextAlignment(.leading)

                HStack(alignment: .bottom) {
                    Text("\(book.bookTotalPages) páginas")
                        .font(.system(.caption, weight: .regular))
                        .foregroundColor(.white.opacity(0.7))

                    Spacer()

                    Text("\(percentageRead)%")
                        .font(.system(.headline, weight: .bold))
                        .foregroundColor(.addNote)
                }
            }
        }
    }

```

```

        }
    }
    .padding(12)
    .frame(maxWidth: .infinity)
    .background(
        Rectangle()
            .fill(.ultraThinMaterial)
            .environment(\.colorScheme, .dark)
    )
}
.contextMenu {
    Button() {
        isEditingSheetPresented = true
    } label: {
        Label("Editar Livro", systemImage: "pencil")

        .font(.body)
    }
    .foregroundColor(.blue)

    Divider()

    Button(role: .destructive) {
        isShowingDeleteAlert = true
    } label: {
        Label("Apagar Livro", systemImage: "trash")
        .font(.body)
    }
    .foregroundColor(.red)
}

.frame(width: 170)
.cornerRadius(12)

.sheet(isPresented: $isEditingSheetPresented, onDismiss: {
    withAnimation{
        bookViewModel.fetchBooks()
    }
}) {
    BookSheetView(bookToEdit: book)
}

.alert("Apagar Livro", isPresented: $isShowingDeleteAlert) {
    Button("Cancelar", role: .cancel) { }

    Button("Apagar", role: .destructive) {
        withAnimation {
            do {
                try bookViewModel.deleteBook(book: book)
            } catch {
                print("Erro ao tentar apagar o livro: \
(error.localizedDescription)")
            }
        }
    }
}

```

```

        }
    }
}
} message: {
    Text("Tem certeza que deseja apagar o livro '\(book.bookTitle ??
"Sem título")'? Essa ação não pode ser desfeita.")
}
}

```

```

} // #Preview { // ZStack { // HStack(spacing: 16) { //
// BookCardView( // book: PreviewProviderHelper.sampleBook // ) // } // } // } //

```

Arquivo: \./View/Components/BooksComponents/CardTotalPages.swift\

swift // // CardTotalPages.swift // CH4-Books // // Created by Julio Sampaio on 18/08/26. //

import Foundation import SwiftUI

struct CardTotalPages: View { let totalPages: Int16

```

var body: some View {

    HStack {
        Image("BookPagesReadCard")
            .resizable()
            .scaledToFit()
            .frame(width: 98, height: 51)

        Spacer()

        VStack (alignment: .leading, spacing: 4){

            if totalPages >= 1 {
                Text("\(totalPages) páginas lidas")
                    .font(.bitter(.bold, style: .title3))
                    .foregroundStyle(Color.black)

                Text(" continue assim!")
                    .font(.bitter(.regular, style: .footnote))
                    .foregroundStyle(Color.black)
            } else {
                Text("Que tal começar agora mesmo?")
                    .font(.bitter(.medium, style: .subheadline))
                    .foregroundStyle(Color.black)
            }
        }

        Spacer()

    }
    .padding(24)
}

```

```

        .background(
            LinearGradient(
                colors: [
                    Color("PagesReadCard2"),
                    Color("PagesReadCard1")
                ],
                startPoint: .leading,
                endPoint: .trailing
            ),
            in: RoundedRectangle(cornerRadius: 20)
        )
    }
}

```

```

}

```

```

#Preview { CardTotalPages(totalPages: 0) }

```

Arquivo: \./View/Components/BooksComponents/ToolBarAddButton.swift\

```

swift // // ToolBarAddButton.swift // CH4-Books // // Created by Julio Sampaio on 16/08/26. //

```

```

// // ToolBarAddButton.swift // CH4-Books // // Created by Julio Sampaio on 16/08/26. //

```

```

import SwiftUI

```

```

struct BookCaseToolbar: ToolbarContent { var onAddClick: () -> Void

```

```

    @ObservedObject var filterViewModel: BookFilterViewModel

    var body: some ToolbarContent {
        ToolbarItem(placement: .topBarTrailing) {
            Button(action: {
                onAddClick()
            }) {
                Image(systemName: "plus")
                    .fontWeight(.semibold)
                    .foregroundColor(.primary)
            }
            .buttonStyle(.borderedProminent)
            .buttonBorderShape(.circle)
            .tint(Color("ActionColor"))
        }

        ToolbarItem(placement: .topBarTrailing) {
            Menu {
                Menu {
                    Picker("Gênero", selection:
$filterViewModel.selectedCategory) {
                        ForEach(filterViewModel.availableCategories, id:
\.self) { category in

```

```

                Text(category).tag(category)
            }
        }
    } label: {
        Label("Gêneros", systemImage: "books.vertical")
    }

    Section("Ordenar por") {
        Picker("Ordenação", selection:
$filterViewModel.currentSort) {
            in
                ForEach(BookSortOption.allCases, id: \.self) { option
                    Text(option.rawValue).tag(option)
                }
            }
        } label: {
            Image(systemName: "ellipsis")
                .fontWeight(.semibold)
                .foregroundColor(.primary)
        }
        .buttonStyle(.plain)
        .buttonBorderShape(.circle)
    }
}

```

```

}

```

```

#Preview { struct PreviewWrapper: View { var body: some View { NavigationStack { Text("Conteúdo da
sua View aqui") .toolbar { BookCaseToolbar( onAddClick: { print("Clicou no add") }, filterViewModel:
BookFilterViewModel() ) } } } }

```

```

return PreviewWrapper()

```

```

}

```

Arquivo: \./View/Components/BooksComponents/DailyGoalCard.swift\

```

swift // // DailyGoalCard.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on
19/08/26. //

```

```

import SwiftUI

```

```

struct DailyGoalCardView: View { @EnvironmentObject var userSettingsViewModel:
UserSettingsViewModel

```

```

var onEditAction: () -> Void

```

```

private var rawProgress: Double {
    let target = Double(userSettingsViewModel.dailyGoal)
    guard target > 0 else { return 0 }
    return Double(userSettingsViewModel.minutesReadToday) / target
}

private var progress: Double {
    min(rawProgress, 1.0)
}

private var isGoalCompleted: Bool {
    let target = userSettingsViewModel.dailyGoal
    return target > 0 && userSettingsViewModel.minutesReadToday >= target
}

var body: some View {
    VStack(alignment: .leading, spacing: 0) {
        HStack {
            HStack(spacing: 8) {
                Image(systemName: isGoalCompleted ? "checkmark.circle.fill"
: "target")
                    .font(.system(.body, weight: .bold))
                    .foregroundColor(isGoalCompleted ? .action : .orange)

                Text("Meta diária de leitura")
                    .font(.system(.body, weight: .regular))
                    .foregroundColor(.white)
            }

            Spacer()

            Button(action: onEditAction) {
                Image(systemName: "pencil")
                    .font(.system(.title2))
                    .fontWeight(.semibold)
                    .foregroundColor(.primary)
                    .padding(6)
            }
                .buttonStyle(.glass)
                .buttonBorderShape(.circle)
                .padding(.trailing, 0)
        }

        HStack(alignment: .firstTextBaseline, spacing: 6) {
            Text("\(userSettingsViewModel.dailyGoal)")
                .font(.bitter(.medium, style: .largeTitle))
                .foregroundColor(.white)

            Text("minutos")
                .font(.bitter(.regular, style: .title3))
                .foregroundColor(.white)
        }
        .padding(.bottom, 4)
    }
}

```

```

        GeometryReader { geometry in
            ZStack(alignment: .leading) {
                Capsule()
                    .fill(Color.white.opacity(0.15))
                    .frame(height: 8)

                Capsule()
                    .fill(
                        LinearGradient(
                            colors: isGoalCompleted ? [.yellow, .orange] :
[.yellow.opacity(0.9), .orange.opacity(0.9)],
                            startPoint: .topLeading,
                            endPoint: .bottomTrailing
                        )
                    )
                    .frame(width: geometry.size.width * CGFloat(progress),
height: 8)
            }
        }
        .frame(height: 8)
        .padding(.vertical, 2)
        .padding(.bottom, 6)

        HStack {
            HStack(spacing: 4) {
                Text("\(userSettingsViewModel.minutesReadToday)")
                    .font(.system(.callout))
                    .foregroundColor(isGoalCompleted ? .action : .orange)

                Text(isGoalCompleted ? "minutos • Meta concluída!" :
"minutos de leitura hoje")
                    .font(.system(.callout))
                    .foregroundColor(isGoalCompleted ? .action.opacity(0.9)
: Color("TextPagesColor"))
            }

            Spacer()

            Text("\(userSettingsViewModel.dailyGoal) min")
                .font(.system(size: 14, weight: .regular))
                .foregroundColor(Color("TextPagesColor"))
        }
    }
    .padding(.horizontal, 18)
    .padding(.vertical, 16)
    .background(
        ZStack {
            RoundedRectangle(cornerRadius: 24)
                .fill(.ultraThinMaterial)

            RoundedRectangle(cornerRadius: 24)
                .fill(Color("DailyGoalCardColor"))
        }
    )

```

```

        .overlay(
            RoundedRectangle(cornerRadius: 24)
                .stroke(

```

```

// .fill(LinearGradient(colors: [.yellow, .orange], startPoint: .leading, endPoint: .trailing)) LinearGradient(
colors: isGoalCompleted ? [.yellow, .orange] : [.white.opacity(0.3), .white.opacity(0.05)], startPoint:
.topLeading, endPoint: .bottomTrailing ), lineWidth: isGoalCompleted ? 1.0 : 0.5 ) ) } }

```

```

#Preview { ZStack { Color("BackgroundColorViews") .ignoresSafeArea()

```

```

        DailyGoalCardView(
            onEditAction: {
                print("Editar meta clicado")
            }
        )
        .environmentObject(UserSettingsViewModel())
        .padding()
    }

```

```

}

```

Arquivo: \./View/Components/BooksDetailsComponents/BooksDetailsToolbar.swift\

swift // // BooksDetailsToolbar.swift // CH4-Books // // Created by Julio Sampaio on 18/08/26. //

```

import Foundation

```

```

import SwiftUI

```

```

struct BooksDetailsToolbar: ToolbarContent {

```

```

    var onEdit: () -> Void

    var body: some ToolbarContent {
        ToolbarItem(placement: .topBarTrailing) {
            Button(action: {
                onEdit()
            }) {
                Image(systemName: "pencil")

                .fontWeight(.semibold)
                .foregroundColor(.primary)
            }
            .buttonStyle(.borderedProminent)
            .buttonBorderShape(.circle)
            .tint(Color("ActionColor"))
        }
    }

```

```
}
```

```
} #Preview { struct PreviewWrapper: View { @State private var showAlert = false
```

```
    var body: some View {
        NavigationStack {
            Text("Conteúdo da sua Sheet aqui")
                .toolbar {
                    BooksDetailsToolbar(onEdit: {})
                }
        }
    }
}

return PreviewWrapper()
```

```
}
```

Arquivo: \./View/Components/BooksDetailsComponents/BookInstanceDetailView.swift\

```
swift // // teste.swift // CH4-Books // // Created by Lucas on 17/08/26. // import SwiftUI
```

```
struct BookInstanceDetailView: View { @ObservedObject var book: Books @EnvironmentObject var
photoLibraryViewModel: PhotoLibraryViewModel
```

```
    var readingProgress: Double {
        guard book.bookTotalPages > 0 else { return 0.0 }
        let current = Double(book.bookCurrentPage)
        let total = Double(book.bookTotalPages)
        return min(max(current / total, 0.0), 1.0)
    }

    var formattedReadingProgress: String {
        let percentage = readingProgress * 100
        return String(format: "%.0f%%", percentage)
    }

    var body: some View {
        VStack(spacing: 16) {

            TitleComponent(title: book.bookTitle ?? "Título desconhecido")

            if let coverData = photoLibraryViewModel.getCoverImage(for: book){
                Image(uiImage: coverData)
                    .resizable()
                    .scaledToFill()
            }
        }
    }
}
```

```

        .frame(width: 200, height: 300)
        .clipShape(RoundedRectangle(cornerRadius: 12))
        .shadow(radius: 5)
    } else{
        ZStack {
            RoundedRectangle(cornerRadius: 12)
                .fill(Color(.systemGray2))

            Image("defaultBook")
                .font(.system(size: 40))
                .foregroundColor(.gray)
        }
        .frame(width: 200, height: 300)
        .shadow(radius: 5)
    }

    Text("Autor: \${book.bookAuthor ?? "Erro"}")
        .font(.body)
        .foregroundColor(.white)

    Text("\${book.bookCategory ?? "erro"}")
        .font(.system(.footnote, weight: .medium))
        .foregroundColor(.black)
        .padding(.horizontal, 40)
        .padding(.vertical, 6)
        .background(Color("TagNoteColor"))
        .opacity(0.8)
        .clipShape(Capsule())

    VStack(spacing: 16) {
        Divider()
            .frame(height: 0.3)
            .background(Color("LinesColor"))
            .padding(.horizontal, 20)

        HStack {
            Spacer()
            VStack(spacing: 4) {
                Text("Início da leitura")

                .font(.footnote).foregroundColor(Color("InfosDetailsView"))
                Text("24/02/2026")
                    .font(.subheadline)
                    .fontWeight(.semibold)
                    .foregroundColor(Color("Texts"))
            }
            Spacer()
            VStack(spacing: 4) {

                Text("Páginas").font(.footnote).foregroundColor(Color("InfosDetailsView"))
                Text("\${book.bookTotalPages} páginas")
                    .font(.subheadline)
                    .fontWeight(.semibold)
                    .foregroundColor(Color("Texts"))
            }
        }
    }

```

```

        }
        Spacer()
        VStack(spacing: 4) {
            Text("Progresso").font(.footnote)
                .foregroundColor(Color("InfosDetailsView"))
            Text("\(formattedReadingProgress)")
                .font(.subheadline)
                .fontWeight(.semibold)
                .foregroundColor(Color("Texts"))
        }
        Spacer()
    }

    Divider()
        .frame(height: 0.3)
        .background(Color("LinesColor"))
        .padding(.horizontal, 20)
    }
    .padding(.bottom, 10)
}
}

```

```

}

```

Arquivo: \./View/Components/BooksDetailsComponents/NotesHeaderview.swift\

```

swift // // NotesHeaderview.swift // CH4-Books // // Created by Lucas on 17/08/26. // import SwiftUI

struct NotesHeaderview: View { @Binding var isPresentedAddNote: Bool var body: some View { HStack {
    Text("Anotações") .font(.bitter(.medium,style: .title3)) .foregroundColor(Color("Texts")) Spacer()
    Button(action: { isPresentedAddNote = true }) { Image(systemName: "plus") .foregroundColor(.white)
        .font(.body) .fontWeight(.medium) .padding(.all, 4) } .buttonStyle(glass) .buttonBorderShape(.circle) }
    // .padding(.horizontal,24) } } #Preview { NotesHeaderview(isPresentedAddNote: .constant(false)) }

```

Arquivo: \./View/Components/SheetsComponents/NoteFieldView.swift\

```

swift // // NoteFieldView.swift // CH4-Books // // Created by Lucas on 17/08/26. //

import SwiftUI // campos para visualizar a nota struct NoteFieldView: View { let note: Notes

```

```

    var body: some View {
        HStack(spacing: 16) {
            RoundedRectangle(cornerRadius: 4)
                .fill(categoryColor(for: note.noteCategory))
                .frame(width: 12, height: 40)

            VStack(alignment: .leading, spacing: 4) {
                Text(note.noteTitle ?? "Sem título")
            }
        }
    }
}

```

```

        .font(.system(size: 16, weight: .bold))
        .foregroundColor(.primary)

        Text(note.noteDescription ?? "")
            .font(.system(size: 14, weight: .regular))
            .foregroundColor(.secondary)
    }

    Spacer()

    Image(systemName: "chevron.right")
        .font(.system(size: 14, weight: .semibold))
        .foregroundColor(.secondary)
    }
    .padding(.horizontal, 16)
    .padding(.vertical, 16)
}

private func categoryColor(for category: String?) -> Color {
    switch category {
    case "Citação": return .pink
    case "Resumo": return .blue
    case "Pensamento": return .yellow
    case "Crítica": return .red
    case "Conceito": return .purple
    case "Lição": return .green
    case "Pergunta": return .orange
    case "Favorito": return .indigo
    default: return .gray
    }
}
}

```

Arquivo: \./View/Components/SheetsComponents/BottomSheetToolbar.swift\

swift // // BottomSheetToolbar.swift // CH4-Books // // Created by Julio Sampaio on 20/08/26. //

import Foundation

import SwiftUI

struct BottomSheetToolbar: ToolbarContent { var title: String var onClose: () -> Void var onEdit: () -> Void

```

var body: some ToolbarContent {
    ToolbarItem(placement: .topBarLeading) {
        Button(action: {
            onClose()
        }) {
            Image(systemName: "xmark")
        }
    }
}

```

```

                .fontWeight(.semibold)
                .foregroundColor(.white)
            }
            .buttonStyle(.plain)
            .buttonBorderShape(.circle)
            .tint(Color.white.opacity(0.2))
        }

        ToolbarItem(placement: .principal) {
            Text(title)
                .foregroundColor(Color("LinesColor"))
                .font(.bitter(.semibold, style: .title2))
        }

        ToolbarItem(placement: .topBarTrailing) {
            Button(action: {
                onEdit()
            }) {
                Image(systemName: "checkmark")
                    .fontWeight(.semibold)
                    .foregroundColor(.primary)
            }

            .buttonStyle(.borderedProminent)
            .buttonBorderShape(.circle)
            .tint(Color("ActionColor"))
        }
    }
}

```

```

}

```

```

#Preview { struct PreviewWrapper: View { @State private var showAlert = false

```

```

    var body: some View {
        NavigationStack {
            Text("Conteúdo da sua Sheet aqui")
                .toolbar {
                    NotesToolBar(
                        title: "Notas", onClose: {}, onEdit: {}
                    )
                }
        }
    }
}

return PreviewWrapper()

```

```

}

```

Arquivo: \./View/Components/SheetsComponents/TextFieldSheets.swift\

swift // // TextFieldSheets.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on 16/08/26. //

import SwiftUI

struct TextFieldSheets: View { @Binding var text: String var placeholder: String var label: String? = nil

```
var body: some View {
    VStack (alignment: .leading){
        if let label = label {
            Text(label)
                .font(.system(.title3, weight: .medium))
                .foregroundColor(Color("LinesColor"))
                .padding(.bottom, 6)
                .padding(.leading, 4)
        }

        TextField("", text: $text, prompt: Text(placeholder)
            .font(.system(.body, weight: .regular))
            .foregroundColor(Color("TextFieldPlaceholderColor"))
        )
        .foregroundColor(.white)
        .padding()
        .font(.system(.body, weight: .regular))
        .overlay(
            RoundedRectangle(cornerRadius: 100)
                .stroke(Color("LinesColor"), lineWidth: 0.5)
        )
    }
}
```

}

#Preview { ZStack { Color.black.ignoresSafeArea()

```
VStack(spacing: 20) {
    TextFieldSheets(
        text: .constant(""),
        placeholder: "Digite aqui...",
        label: "Título do Campo"
    )

    TextFieldSheets(
        text: .constant(""),
        placeholder: "Apenas com placeholder"
```

```

        )
    }
    .padding()
}

```

```

}

```

Arquivo: \./View/Components/SheetsComponents/MenuSheetPicker.swift\

swift // // MenuSheetPicker.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on 16/08/26. //

import SwiftUI

struct MenuSheetPicker: View { var title: String? = nil var placeholder: String @Binding var selectedValue: String var options: [String] var formatOption: (String) -> String = { \$0 }

```

var body: some View {
    VStack(alignment: .leading, spacing: 6) {
        if let title = title, !title.isEmpty {
            Text(title)
                .font(.system(.title3, weight: .medium))
                .foregroundColor(Color("LinesColor"))
                .padding(.leading, 4)
        }

        Menu {
            ForEach(options, id: \.self) { option in
                Button(formatOption(option)) {
                    selectedValue = option
                }
            }
        } label: {
            HStack {
                Text(selectedValue.isEmpty ? placeholder :
formatOption(selectedValue))
                    .font(.system(.body, weight: .regular))
                    .foregroundColor(selectedValue.isEmpty ?
Color("TextFieldPlaceholderColor") : .white)

                Spacer()

                Image(systemName: "chevron.up.chevron.down")
                    .font(.footnote.weight(.semibold))
                    .foregroundColor(Color("LinesColor"))
            }
        } .padding()
        .contentShape(Rectangle())
        .overlay(
            RoundedRectangle(cornerRadius: 100)

```

```

        .stroke(Color("LinesColor"), lineWidth: 0.5)
    )
}
.buttonStyle(.plain)
}
}
}

```

Arquivo: \./View/Components/SheetsComponents/MediaThumbView.swift\

swift // // MediaThumbView.swift // CH4-Books // // Created by Julio Sampaio on 17/08/26. //

import Foundation import SwiftUI

struct MediaThumbnailView: View { let image: UIImage let onDelete: () -> Void

```

var body: some View {
    ZStack(alignment: .topTrailing) {
        Image(uiImage: image)
            .resizable()
            .scaledToFill()
            .frame(width: 120, height: 120)
            .clipped()
            .cornerRadius(10)

        Button(action: onDelete) {
            Image(systemName: "xmark.circle.fill")
                .foregroundColor(.white)
                .background(Color.black.opacity(0.6))
                .clipShape(Circle())
                .padding(6)
        }
    }
}
}
}

```

Arquivo: \./View/Components/SheetsComponents/NotesToolBar.swift\

swift // // NotesHeaderEditView.swift // CH4-Books // // Created by Lucas on 18/08/26. //

import SwiftUI

struct NotesToolBar: ToolbarContent { var title: String var onClose: () -> Void var onEdit: () -> Void

```

var body: some ToolbarContent {
    ToolbarItem(placement: .topBarLeading) {
        Button(action: {
            onClose()
        }) {
            Image(systemName: "chevron.left")
                .fontWeight(.semibold)
                .foregroundColor(.white)
        }
        .buttonStyle(.plain)
        .buttonBorderShape(.circle)
        .tint(Color.white.opacity(0.2))
    }

    ToolbarItem(placement: .principal) {
        Text(title)
            .foregroundColor(Color("LinesColor"))
            .font(.bitter(.semibold, style: .title2))
    }

    ToolbarItem(placement: .topBarTrailing) {
        Button(action: {
            onEdit()
        }) {
            Image(systemName: "pencil")
                .fontWeight(.semibold)
                .foregroundColor(.primary)

        }

        .buttonStyle(.borderedProminent)
        .buttonBorderShape(.circle)
        .tint(Color("ActionColor"))
    }
}

```

```

}

```

```

#Preview { struct PreviewWrapper: View { @State private var showAlert = false

```

```

var body: some View {
    NavigationStack {
        Text("Conteúdo da sua Sheet aqui")
            .toolbar {
                NotesToolBar(
                    title: "Notas", onClose: {}, onEdit: {}
                )
            }
    }
}

```

```
}

return PreviewWrapper()

}
```

Arquivo: \./View/Components/SheetsComponents/CategorySelectGroup.swift\

swift // // CategorySelectGroup.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on 16/08/26. //

import SwiftUI

struct CategoryMenuView: View { var title: String //let categories = ["Citação", "Resumo", "Pensamento", "Crítica"] let categories: [String] @Binding var selectedCategory: String

```
var body: some View {
    VStack(alignment: .leading, spacing: 8) {
        Text(title)
            .font(.system(.title3, weight: .medium))
            .foregroundColor(Color("LinesColor"))
            .padding(.bottom, 6)
            .padding(.leading, 4)

        Menu {
            Picker("Categoria", selection: $selectedCategory) {
                ForEach(categories, id: \.self) { category in
                    Text(category).tag(category)
                }
            } label: {
                HStack {
                    Text(selectedCategory.isEmpty ? "Selecione uma categoria" :
selectedCategory)
                        .font(.system(.body, weight: .regular))
                        .foregroundColor(selectedCategory.isEmpty ?
Color("TextFieldPlaceholderColor2") : .white)

                    Spacer()

                    Image(systemName: "chevron.up.chevron.down")
                        .font(.footnote.weight(.semibold))
                        .foregroundColor(Color("TextFieldPlaceholderColor2"))
                }
            }.padding()
                .contentShape(Rectangle())
                .overlay(
                    RoundedRectangle(cornerRadius: 100)
                        .stroke(Color("LinesColor"), lineWidth: 0.5)
                )
    }
}
```

```

        }
        .buttonStyle(.plain)
    }
}

```

```

}

#Preview { ZStack { Color.gray.opacity(0.2).ignoresSafeArea() CategoryMenuView( title: "Escolher
categoria", categories: ["Citação", "Resumo", "Pensamento", "Crítica"], selectedCategory:
.constant("Teste"), //selectedCategory: .constant("Citação"), ) .padding() } }

```

Arquivo: \./View/Components/SheetsComponents/NotesSectionView.swift\

swift // // NotesSectionView.swift // CH4-Books // // Created by Lucas on 17/08/26. //

import SwiftUI import CoreData

struct NotesSectionView: View { var notes: [Notes]

```

    var onNotesChanged: () -> Void = { }

    @EnvironmentObject private var notesViewModel: NotesViewModel

    @State private var selectedNote: Notes?

    var body: some View {
        Group {
            if notes.isEmpty {
                Text("Adicione sua primeira anotação")
                    .font(.subheadline)
                    .foregroundColor(Color("Texts"))
                    .padding(.top, 20)
                    .padding(.bottom, 10)
            } else {
                LazyVStack() {
                    NavigationStack{
                        ForEach(notes, id: \.objectID) { note in
                            NavigationLink(destination:
                                NoteDetailView(note: note)){
                                    NoteRowView(note: note)
                                }
                        }
                    }
                }
            }
        }
        .buttonStyle(.plain)
        .padding(.vertical, 10)
        .padding(.horizontal, 28)

        if note.objectID != notes.last?.objectID {

```

}

88 / 114

```

        .foregroundColor(Color("Texts"))
        .lineLimit(1)

        Text(note.noteDescription ?? "Sem conteúdo")
            .font(.subheadline)
            .foregroundColor(Color("Texts"))
            .lineLimit(1)
    }

    Spacer()

    Image(systemName: "chevron.right")
        .foregroundColor(Color("Texts"))
        .font(.body)
        .fontWeight(.medium)
    }
    .contentShape(Rectangle())
}

```

```

}

#Preview("Com Anotações") { // 1. Instância do CoreData em memória para o Preview // NOTA: Altere
"CH4_Books" para o nome exato do seu arquivo .xcdatamodeld let container =
NSPersistentContainer(name: "CH4_Books") container.persistentStoreDescriptions.first?.url =
URL(fileURLWithPath: "/dev/null") container.loadPersistentStores { _, error in if let error = error {
fatalError("Erro no CoreData Preview: (error.localizedDescription)") } }

```

```

let context = container.viewContext

// 2. Dados mockados (Mock Notes)
let note1 = Notes(context: context)
note1.noteTitle = "Anotações do Capítulo 1"
note1.noteDescription = "Principais insights sobre a introdução do livro e
conceitos base."

let note2 = Notes(context: context)
note2.noteTitle = "Citação Favorita"
note2.noteDescription = "A simplicidade é o último grau de sofisticação."

return NavigationStack {
    ScrollView {
        NotesSectionView(notes: [note1, note2])
            .padding()
    }
    .background(Color("BackgroundColorViews"))
}
.environment(\.managedObjectContext, context)
.environmentObject(NotesViewModel())

```

```
}

#Preview("Lista Vazia") {

    NavigationStack {
        // .ignoresSafeArea()

        NotesSectionView(notes: [])
            .padding()
            .background(Color(uiColor: .systemGroupedBackground))
    }
    .environmentObject(NotesViewModel())
}
```

Arquivo: \./View/Components/SheetsComponents/SheetHeaderView.swift\

swift // // SheetHeaderView.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on 14/08/26. //

import SwiftUI

struct SheetHeaderView: ToolbarContent { let title: String let actionIcon: String let hasChanges: Bool

```
@Binding var showingDiscardAlert: Bool

var onCancel: () -> Void
var onConfirm: () -> Void
var onDiscard: () -> Void

var body: some ToolbarContent {
    ToolbarItem(placement: .topBarLeading) {
        Button(action: {
            if hasChanges {
                showingDiscardAlert = true
            } else {
                onDiscard()
            }
        }) {
            Image(systemName: "xmark")
                .fontWeight(.semibold)
                .foregroundColor(.primary)
        }
        .confirmationDialog(
            "Atenção",
            isPresented: $showingDiscardAlert,
            titleVisibility: .hidden
        ) {
            Button("Descartar", role: .destructive) {
```

```

        onDiscard()
    }

    Button("Continuar Editando", role: .cancel) { }

    } message: {
        Text("Deseja mesmo descartar a edição?")
    }
}

ToolBarItem(placement: .principal) {
    Text(title)
        .foregroundColor(Color("LinesColor"))
        .font(.bitter(.semibold, style: .title2))
}

ToolBarItem(placement: .topBarTrailing) {
    Button(action: {
        onConfirm()
    }) {
        Image(systemName: actionIcon)
            .fontWeight(.semibold)
            .font(.body.bold())
            .foregroundColor(.title)
    }
    .buttonStyle(.borderedProminent)
    .buttonBorderShape(.circle)
    .tint(Color("ActionColor"))
}
}
}

```

```

} #Preview { struct PreviewWrapper: View { @State private var showAlert = false

```

```

var body: some View {
    NavigationStack {
        Text("Conteúdo da sua Sheet aqui")
        .toolbar {
            SheetHeaderView (
                title: "Cadastrar livro",
                actionIcon: "checkmark",
                hasChanges: false,
                showingDiscardAlert: $showAlert,
                onCancel: { print("Clicou no X") },
                onConfirm: { print("Clicou no Check") },
                onDiscard: { print("Clicou em descartar") }
            )
        }
    }
}
}
}
}

```

```
return PreviewWrapper()
```

```
}
```

Arquivo: \./View/Components/GeralComponents/ButtonAction.swift\

```
swift // // ButtonAction.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on  
19/08/26. //
```

```
import SwiftUI
```

```
struct ButtonAction: View { var text: String var colorButton: String? = nil var isGlass: Bool = false var  
action: (() -> Void)? = nil
```

```
    var body: some View {  
        Button(action: {  
            action?()  
        }) {  
            Text(text)  
                .font(.body)  
                .frame(maxWidth: .infinity)  
                .fontWeight(.medium)  
                .padding(.vertical, isGlass ? 8 : 14)  
                .background(  
                    Group {  
                        if let colorName = colorButton, !isGlass {  
                            Color(colorName)  
                        } else {  
                            Color.clear  
                        }  
                    }  
                )  
                .foregroundColor(.white)  
                .clipShape(Capsule())  
        }  
        .frame(height: 50) // Garante a mesma altura padrão para todos  
        .if(isGlass) { view in  
            view.buttonStyle(.glass)  
        }  
    }  
}
```

```
}
```

```
extension View { @ViewBuilder func if<Content: View>(_ condition: Bool, transform: (Self) -> Content) -  
> some View { if condition { transform(self) } else { self } }
```

```
#Preview { VStack(spacing: 16) { // Button com .buttonStyle(.glass) ButtonAction(text: "Excluir", isGlass: true) { print("Excluído") }
```

```
        // Button preenchido com cor
        ButtonAction(text: "Salvar", colorButton: "ActionColor")
    }
    .padding()
```

```
}
```

Arquivo: \./View/Components/GeralComponents/TipsComponent.swift\

```
swift // // TipsComponent.swift // CH4-Books // // Created by Julio Sampaio on 16/08/26. //
```

```
import Foundation import SwiftUI
```

```
struct TipsComponent: View { let content: String
```

```
    var body: some View {
        Text(content)
            .font(.caption)
            .fontWeight(.regular)
            .foregroundColor(.secondary)
            .padding(.horizontal, 5)
    }
```

```
}
```

```
#Preview { TipsComponent( content: "Tips" ) }
```

Arquivo: \./View/Components/GeralComponents/ToolBarButton.swift\

```
swift // // StopwatchToolbar.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on 19/08/26. //
```

```
// // StopwatchToolbar.swift // CH4-Books //
```

```
import SwiftUI
```

```
struct ToolBarButton: ToolbarContent { var action: () -> Void var icon: String var colorName: String?
```

```
    var body: some ToolbarContent {
        ToolbarItem(placement: .topBarTrailing) {

            Button(action: {
                action()
            }) {
```

```

        Image(systemName: icon)
            .font(icon == "square.and.pencil" ? .body : .body)
            .fontWeight(.semibold)
            .foregroundColor(.primary)
            .padding(.bottom, icon == "square.and.pencil" ? 5 : 0)
    }
    .buttonStyle(.borderedProminent)
    .buttonBorderShape(.circle)
    .tint(colorName != nil ? Color(colorName!) : Color(""))
}
}
}

```

```

}

```

```

#Preview { NavigationStack { Text("Preview") .toolbar { ToolBarButton(action: {}, icon:
"square.and.pencil") } } .preferredColorScheme(.dark) }

```

Arquivo: \./View/Notes/MyNotesListView.swift\

```

swift // // NoteListView.swift // CH4-Books // // Created by Lucas on 19/08/26. //

```

```

import SwiftUI import CoreData

```

```

struct MyNotesListView: View { @StateObject private var viewModel = NotesViewModel()
@Environment(.dismiss) var dismiss @State private var selectedNote: Notes? @State private var
isPresentedAddNote: Bool = false

```

```

var book: Books?

var body: some View {
    NavigationStack{
        ZStack {
            Color(.backgroundColorViews).ignoresSafeArea()
            VStack(alignment: .leading, spacing: 20) {
                TitleComponent(title: "Minhas notas")
                    .padding(.horizontal, 24)
                    .padding(.top, 10)

                ScrollView(showsIndicators: false) {
                    LazyVStack(spacing: 24) {
                        ForEach(viewModel.savedNotes, id: \.self) { note in
                            NavigationLink(destination:
                                NoteDetailView(note: note)){
                                    NoteRowView(note: note)
                                }
                        }
                    }
                }
            }
        }
    }
}

```

```

        }
    }
    .padding(.horizontal)
    .padding(.bottom, 30)
}
}
}
.navigationBarBackButtonHidden(true)
.toolbar {
    MyNotesToolBar(onBackClick: { dismiss() }, onAddClick: {
        isPresentedAddNote = true
    })
}
.onAppear {
    viewModel.fetchNotes(for: book)
}
//      .sheet(item: $selectedNote) { note in
//          NoteDetailSheetView(note: note)
//      }
.sheet(isPresented: $isPresentedAddNote, onDismiss: {
    viewModel.fetchNotes(for: book)
}) {
    if let book = book {
        NoteSheetView(book: book)
        .environmentObject(viewModel)
    }
}
}
}
```

```
struct NotesView: View {
```

```

var body: some View {
    NavigationView {

        ZStack {
            Color("BackgroundColorViews")
                .ignoresSafeArea()

            VStack(alignment: .leading, spacing: 20) {
                Text("Minhas notas")
                    .font(.bitter(.medium, style: .largeTitle))
                    .foregroundColor(Color("TitleColor"))

                ScrollView(showsIndicators: false) {
                    VStack(spacing: 16) {
                        ForEach(0..<5) { _ in
                            NotesCardView(
                                imageName: "defaultBook",
                                tagText: "Referência",
                                title: "Título da nota",
                                description: "Descrição inicial da
primeira..."
                            )
                        }
                    }
                }
            }
        }
        .padding(.horizontal, 24)

    }
    .navigationBarTitleDisplayMode(.inline)
    .toolbar {
        ToolbarButton(action: {}, icon: "plus", colorName:
"ActionColor")
    }
}
}

```

```

}

```

```

#Preview { NotesView() }

```

Arquivo: \./View/Stopwatch/StopwatchView.swift\

```

swift // // StopwatchView.swift // CH4-Books // // Created by Lucas on 15/08/26. //

```

```

import SwiftUI

```

```

struct StopwatchView: View { @State var selectedBook: Books @State private var isShowingSheet = false
@EnvironmentObject var stopwatchViewModel: StopwatchViewModel //@State private var progress:

```

Double = 0.5

```

var body: some View {

    GeometryReader { geometry in
        let cardWidth = geometry.size.width * 0.49
        let cardHeight = geometry.size.height * 0.38

        ZStack {
            Color("BackgroundColorViews")
                .ignoresSafeArea()

            // 10 Anéis Concentricos ao Fundo
            ConcentricRingsView(progress:
Double(stopwatchViewModel.getBookProgress(book: selectedBook)))
                .ignoresSafeArea()

            VStack {
                Spacer()

                //card central
                VStack (spacing: 24){

                    Text(stopwatchViewModel.timerFormatter())
                        .font(.custom("Bitter", size: 70, relativeTo:
.largeTitle))

                        .fontWeight(.medium)
                        .lineLimit(1)
                        .minimumScaleFactor(0.5)
                        .monospacedDigit()

                    //progresso do livro
                    VStack(spacing: 6) {
                        HStack(spacing: 6) {
                            Text("Progresso do livro")
                                .font(.subheadline)
                                .foregroundColor(.white)
                                .fontWeight(.regular)

                            Text("\
(stopwatchViewModel.getBookProgress(book: selectedBook) * 100)%")
                                .font(.subheadline)
                                .fontWeight(.bold)
                                .foregroundColor(Color("ProgressBar"))
                        }

                        ProgressView(value:
stopwatchViewModel.getBookProgress(book: selectedBook))
                            .tint(Color("ProgressBar"))
                            .frame(width: 170)
                    }
                }
            }
        }
    }
}

```

```

//seleção do livro
VStack(spacing: 6) {
    Text("Selecione o livro")
        .font(.subheadline)
        .fontWeight(.regular)
        .foregroundColor(.white)

    Menu {
        Picker("Selecione o livro", selection:
$selectedBook){
            Text("Livro 1").tag("Livro 1")
            Text("Livro 2").tag("Livro 2")
            Text("Livro 3").tag("Livro 3")
        }
    } label: {

```

```

// Text(verbatim: selectedBook.bookTitle) // .font(.caption) // .fontWeight(.medium) //
.foregroundColor(.white.opacity(0.7)) // .padding(.horizontal, 24) // .padding(.vertical, 8) //
.background(Color.white.opacity(0.12)) // .clipShape(Capsule()) }

```

```

//botao inciar
Button(action: {

    stopwatchViewModel.isRunning.toggle()

    if stopwatchViewModel.isRunning {
        stopwatchViewModel.startTimer()
    }
    else{
        stopwatchViewModel.stop()
    }

}) {
    Text(stopwatchViewModel.isRunning ? "Parar" :
"Iniciar")

        .font(.title3)
        .frame(maxWidth: .infinity)
        .fontWeight(.medium)
        .padding(.vertical, 14)
        .background(
            Group {
                if stopwatchViewModel.isRunning {
                    Color.red.opacity(0.85)
                } else {
                    Color("ActionColor")
                }
            }
        )
        .foregroundColor(.white)

```

```

        .clipShape(Capsule())
    }
    .padding(.horizontal, 16)
    .padding(.top, 10)

    }
}
.frame(width: cardWidth, height: cardHeight)
.background(
    RoundedRectangle(cornerRadius: 64, style: .continuous)
        .fill(Color("BackgroundColorViews"))
)
.padding(.bottom, 26)
Spacer()

}
.padding(.horizontal)
}
//botao de editar (top bar)
.overlay(
    HStack {
        Button(action: {
            isShowingSheet.toggle()
        }) {
            Image(systemName: "square.and.pencil")
                .font(.system(size: 20, weight: .semibold))
                .foregroundColor(Color("TitleColor"))
                .frame(width: 48, height: 48)
                .background(Color("ActionColor").opacity(0.8), in:
Circle())
        }
        .glassEffect(.regular, in: Circle())
        .sheet(isPresented: $isShowingSheet) {
            NoteSheetView(book: PreviewProviderHelper.sampleBook)
        }
    }
    .padding(.horizontal, 24)
    .padding(.top, geometry.safeAreaInsets.top > 0 ? 8 : 16),
    alignment: .topTrailing
)

}
}

```

```

}

##Preview { // StopwatchView() // .preferredColorScheme(.dark) //
.environmentObject(StopwatchViewModel()) // .environmentObject(PhotoLibraryViewModel()) //}

```

```
swift // // StopwatchInitialView.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on 19/08/26. //
```

```
import SwiftUI
```

```
struct StopwatchInitialView: View { var namespace: Namespace.ID @EnvironmentObject var stopwatchViewModel: StopwatchViewModel @EnvironmentObject var photoLibraryViewModel: PhotoLibraryViewModel @EnvironmentObject var booksViewModel: BooksViewModel @EnvironmentObject var notesViewModel: NotesViewModel @EnvironmentObject var userSettingsViewModel: UserSettingsViewModel
```

```
@State var selectedBook: Books?

@State private var isShowingNoteSheet = false
@State private var isShowingSelectBookSheet = false
@State private var isShowingTimerPicker = false
@State private var isShowingNoBookAlert = false
@State private var showAbandonAlert = false
@State private var tempReadedPages: String = ""

@State private var showErrorAlert = false
@State private var errorMessage = ""

@State private var selectedDuration: TimeInterval = 15 * 60

private var buttonTitle: String {
    switch stopwatchViewModel.timerState {
    case .running:
        return "Pausar leitura"
    case .paused:
        return "Continuar leitura"
    case .stopped:
        return "Iniciar leitura"
    }
}

var body: some View {
    NavigationStack {
        GeometryReader { geometry in
            ZStack {
                Color("BackgroundColorViews")
                    .ignoresSafeArea()

                backgroundWithGradient(geometry)

                VStack(spacing: 20) {
                    Spacer()

                    // Campo de Seleção do Tempo
                    VStack(spacing: 8) {
                        Text("Selecione o tempo de leitura")
                            .font(.body)
```

```

        .foregroundColor(.white)

        Button(action: {
            isShowingTimerPicker = true
        }) {
            Text(stopwatchViewModel.timerFormater())
                .font(.custom("Bitter", size: 60,
relativeTo: .largeTitle))
                .fontWeight(.semibold)
                .foregroundColor(.white)
                .monospacedDigit()
                .matchedGeometryEffect(id: "timerText", in:
namespace)
                .padding(.horizontal, 12)
                .padding(.vertical, 4)
                .background(
                    ZStack {
                        RoundedRectangle(cornerRadius: 100,
style: .continuous)
                        .fill(Color("StopwatchSelectors").opacity(0.2))
                    }
                )
                .overlay(
                    RoundedRectangle(cornerRadius: 100,
style: .continuous)
                        .stroke(
                            LinearGradient(
                                colors:
[Color.white.opacity(0.6), Color.white.opacity(0.1)],
                                startPoint: .topLeading,
                                endPoint: .bottomTrailing
                            ),
                            lineWidth: 0.5
                        )
                )
                .shadow(color: Color.black.opacity(0.15),
radius: 10, x: 0, y: 5)
        }
        .disabled(stopwatchViewModel.timerState !=
.stopped)
        .buttonStyle(.plain)
    }

    VStack(spacing: 24) {
        bookSelectionButton

        if let existingBook = selectedBook {
            bookProgressSection(existingBook)
        }

        VStack(spacing: 12) {
            ButtonAction(text: buttonText, colorButton:
"ActionColor"){

```

```

        guard selectedBook != nil &&
stopwatchViewModel.timerFormater() != "00:00" else {
            isShowingNoBookAlert = true
            return
        }

        if stopwatchViewModel.timerState ==

.running {

            stopwatchViewModel.pauseTimer()
        } else {
            stopwatchViewModel.startTimer()
        }
    }
    .padding(.top, 8)

    if stopwatchViewModel.timerState != .stopped {
        Button(action: {
            showAbandonAlert = true
        }) {
            Text("Abandonar leitura")

.font(.subheadline.weight(.semibold))
                .foregroundColor(.red)
                .padding(.vertical, 8)
            }
        }
    }
    .padding(.horizontal)
}

Spacer()
}
}
.toolbar {
    ToolbarButton(
        action: {
            if selectedBook != nil {
                isShowingNoteSheet = true
            } else {
                isShowingNoBookAlert = true
            }
        },
        icon: "square.and.pencil"
    )
}

.sheet(isPresented: $isShowingNoteSheet) {
    if let currentBook = selectedBook {
        NoteSheetView(book: currentBook)
    }
}

.alert("Atenção", isPresented: $isShowingNoBookAlert) {
    Button("OK", role: .cancel) { }
}

```

```

    } message: {
        Text("Selecione um livro e um tempo de leitura.")
    }

    .alert("Abandonar leitura", isPresented: $showAbandonAlert) {
        Button("Cancelar", role: .cancel) { }
        Button("Abandonar", role: .destructive) {
            stopwatchViewModel.abandonTimer()
        }
    } message: {
        Text("Tem certeza que deseja abandonar a leitura? O
progresso será perdido.")
    }

    .sheet(isPresented: $isShowingSelectBookSheet) {
        SelectBookSheetView(selectedBook: self.$selectedBook)
        .environmentObject(self.booksViewModel)
    }

    .sheet(isPresented: $isShowingTimerPicker) {
        timerPickerSheet
    }

    .sheet(isPresented: $stopwatchViewModel.showProgressSheet) {
        progressSheet
    }

    .alert("Erro ao executar a ação.", isPresented:
$showErrorAlert) {
        Button("Tentar novamente", role: .cancel) { }
    } message: {
        Text(errorMessage)
    }

    .onAppear {
        booksViewModel.fetchBooks()
        userSettingsViewModel.fetchUserSettings()

        if selectedBook == nil {
            selectedBook = userSettingsViewModel.lastBookReaded
        }

        if let safeBook = selectedBook {
            stopwatchViewModel.getTotalPages(book: safeBook)
            stopwatchViewModel.getCurrentPage(book: safeBook)
        }
    }
    //utilizado para atualizar o cronometro caso haja modificacoes
    .onChange(of: selectedBook) { newBook in
        if let safeBook = newBook {
            stopwatchViewModel.getTotalPages(book:
safeBook)

            stopwatchViewModel.getCurrentPage(book:
safeBook)

```

```

        }
    }
}

private var bookSelectionButton: some View {
    VStack(spacing: 8) {
        Text("Selecione o livro")
            .font(.body)
            .foregroundColor(.white)

        Button(action: {
            isShowingSelectBookSheet = true
        }) {
            HStack(spacing: 12) {
                if let imageData = selectedBook?.bookCover, let uiImage =
UIImage(data: imageData) {
                    Image(uiImage: uiImage)
                        .resizable()
                        .scaledToFill()
                        .frame(width: 40, height: 52)
                        .clipShape(RoundedRectangle(cornerRadius: 8))
                } else {
                    Image("defaultBook")
                        .resizable()
                        .scaledToFit()
                        .frame(width: 40, height: 52)
                        .clipShape(RoundedRectangle(cornerRadius: 8))
                        .padding(.leading, 8)
                }

                VStack(alignment: .leading, spacing: 3) {
                    Text(selectedBook?.bookTitle ?? "Nenhum livro
selecionado")
                        .font(.body)
                        .fontWeight(.semibold)
                        .foregroundColor(.white)
                        .lineLimit(1)

                    Text(selectedBook?.bookAuthor ?? "Toque para
selecionar")
                        .font(.system(size: 13))
                        .foregroundColor(Color.white.opacity(0.8))
                        .lineLimit(1)
                }

                if stopwatchViewModel.timerState == .stopped {
                    Spacer()
                    Image(systemName: "chevron.right")
                        .fontWeight(.semibold)
                        .foregroundColor(.white)
                        .padding(.horizontal, 8)
                }
            }
        }
    }
}

```

```

        }
        .padding(.horizontal, 14)
        .padding(.vertical, 10)
        .background(
            ZStack {
                RoundedRectangle(cornerRadius: 100, style: .continuous)
                    .fill(Color("StopwatchSelectors").opacity(0.20))
            }
        )
        .overlay(
            RoundedRectangle(cornerRadius: 100, style: .continuous)
                .stroke(
                    LinearGradient(
                        colors: [Color.white.opacity(0.6),
Color.white.opacity(0.1)],
                        startPoint: .topLeading,
                        endPoint: .bottomTrailing
                    ),
                    lineWidth: 0.5
                )
        )
        .shadow(color: Color.black.opacity(0.15), radius: 10, x: 0, y:
5)
    }
    .disabled(stopwatchViewModel.timerState != .stopped)
}
.padding(.horizontal, 24)
}

private func bookProgressSection(_ book: Books) -> some View {
    VStack(spacing: 10) {
        HStack {
            Text("Progresso do livro")
                .font(.body)
                .foregroundColor(.white)
            Text("\(Int(stopwatchViewModel.getBookProgress(book: book) *
100))%")
                .font(.body)
                .fontWeight(.bold)
                .foregroundColor(Color("ActionColor"))
        }

        GeometryReader { barGeo in
            ZStack(alignment: .leading) {
                Capsule()
                    .fill(Color.white.opacity(0.15))
                Capsule()
                    .fill(LinearGradient(colors: [.yellow, .orange],
startPoint: .leading, endPoint: .trailing))
                    .frame(width: barGeo.size.width *
CGFloat(stopwatchViewModel.getBookProgress(book: book)))
            }
        }
        .frame(height: 10)
    }
}

```

```

        .padding(.horizontal, 28)
    }
}

private func backgroundWithGradient(_ geometry: GeometryProxy) -> some View
{
    Group {
        if let imageData = selectedBook?.bookCover, let uiImage =
UIImage(data: imageData) {
            Image(uiImage: uiImage)
                .resizable()
        } else {
            Image("defaultBook")
                .resizable()
        }
    }
    .scaledToFill()
    .frame(width: geometry.size.width, height: geometry.size.height)
    .opacity(0.40)
    .overlay(
        ZStack {
            Color("BackgroundColorViews").opacity(0.70)
            LinearGradient(
                colors: [
                    Color.clear,
                    Color("BackgroundColorViews").opacity(0.80),
                    Color("BackgroundColorViews")
                ],
                startPoint: .center,
                endPoint: .bottom
            )
        }
    )
    .clipped()
    .ignoresSafeArea()
}

private var timerPickerSheet: some View {
    VStack(spacing: 20) {
        HStack(spacing: 0) {
            Picker("Horas", selection: Binding(
                get: { Int(selectedDuration) / 3600 },
                set: { newHours in
                    let currentMinutes = (Int(selectedDuration) % 3600) /
60

                    let hoursInSeconds = Double(newHours * 3600)
                    let minutesInSeconds = Double(currentMinutes * 60)
                    let newTotal = hoursInSeconds + minutesInSeconds

                    selectedDuration = newTotal
                    stopwatchViewModel.totalTime = newTotal
                    stopwatchViewModel.elapsedTime = newTotal
                }) {
        }) {

```

```

        ForEach(0..<24, id: \.self) { hour in
            Text("\(hour) h").tag(hour)
        }
    }
    .pickerStyle(.wheel)

    Picker("Minutos", selection: Binding(
        get: { (Int(selectedDuration) % 3600) / 60 },
        set: { newMinutes in
            let currentHours = Int(selectedDuration) / 3600
            let hoursInSeconds = Double(currentHours * 3600)
            let minutesInSeconds = Double(newMinutes * 60)
            let newTotal = hoursInSeconds + minutesInSeconds

            selectedDuration = newTotal
            stopwatchViewModel.totalTime = newTotal
            stopwatchViewModel.elapsedTime = newTotal
        }
    )) {
        ForEach(0..<60, id: \.self) { minute in
            Text("\(minute) min").tag(minute)
        }
    }
    .pickerStyle(.wheel)
}
.padding(.horizontal)

Button("Confirmar") {
    isShowingTimerPicker = false
}
.font(.body.weight(.semibold))
.foregroundColor(Color("ActionColor"))
.padding(.bottom)
}
.presentationDetents([.height(260)])
}

private var progressSheet: some View {
    Group {
        if let currentBook = selectedBook {
            BottomSheet(
                minutesPerDay: .constant(0),
                isPickerShown: false,
                readedPages: $tempReadedPages,
                onDismiss: {
                    stopwatchViewModel.showProgressSheet = false
                },
                onSave: {
                    if let newPage = Int16(tempReadedPages) {
                        currentBook.bookCurrentPage = newPage

                        do {
                            try booksViewModel.saveBook()
                        } catch let error as LocalizedError {

```

```

                errorMessage = error.errorDescription ??
"Ocorreu um erro desconhecido."
                showErrorAlert = true
            } catch {
                errorMessage = "Erro inesperado."
                showErrorAlert = true
            }
        }

        let rawMinutes = Int(stopwatchViewModel.totalTime / 60)
        let minutesRead = max(1, rawMinutes)

        userSettingsViewModel.addCompletedReadingTime(minutes:
minutesRead)

        booksViewModel.fetchBooks()
        userSettingsViewModel.fetchUserSettings()
        tempReadedPages = ""
        stopwatchViewModel.showProgressSheet = false
    }
)
.presentationDetents([.height(340)])
.presentationDragIndicator(.visible)
.presentationBackground(Color("BackgroundColorViews"))
    }
}
}

```

```

}

```

Arquivo: \./View/Onboarding/FirstOnboardView.swift\

swift // // FirstOnboardView.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on 17/08/26. //

```

import SwiftUI

```

```

struct FirstOnboardView: View { var onAdvance: (() -> Void)? = nil

```

```

    var body: some View {
        ZStack {
            Color("BackgroundColorViews")
                .ignoresSafeArea()

            Image("OnboardingOficial")
                .resizable()
                .scaledToFit()
                .frame(maxWidth: .infinity, maxHeight: .infinity, alignment:
.top)

                .mask(
                    LinearGradient(

```

```

        gradient: Gradient(stops: [
            .init(color: .black, location: 0.68),
            .init(color: .clear, location: 0.52)
        ]),
        startPoint: .top,
        endPoint: .bottom
    )
)

VStack(spacing: 16) {
    Spacer()

    Capsule()
        .fill(Color("ProgressBar"))
        .frame(width: 36, height: 4)
        .padding(.top, 8)

    VStack(spacing: 2) {
        Text("Bem-vindo(a)")
            .font(.largeTitle)
            .fontWeight(.regular)
            .foregroundColor(.white)

        HStack(spacing: 6) {
            Text("ao")
                .font(.largeTitle)
                .fontWeight(.regular)
                .foregroundColor(.white)

            Text("Frush")
                .font(.bitter(.bold, style: .largeTitle))
                .foregroundColor(.white)
        }
    }

    // Subtítulo com destaque final em amarelo
    (Text("Venha cadastrar seus livros, registrar suas notas de
    leitura e alcançar suas metas ")
        .foregroundColor(.white.opacity(0.85)) +
        Text("no seu próprio ritmo.")
            .foregroundColor(Color("ProgressBar"))
            .bold())
        .font(.body)
        .multilineTextAlignment(.center)
        .padding(.horizontal, 16)

    Spacer()

    ButtonAction(text: "Avançar", colorButton: "ActionColor"){
        onAdvance?()
    }
    .padding(.bottom, 40)

```

```

        }
        .padding(.horizontal, 26)
        .padding(.top, 420)
    }
}

```

```

}

```

```

#Preview { FirstOnboardView() }

```

Arquivo: \./View/Onboarding/SecondOnboardView.swift\

```

swift // // SecondOnboardView.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on
17/08/26. // /* import SwiftUI

```

```

struct SecondOnboardView: View { @EnvironmentObject var booksViewModel: BooksViewModel @State
private var selectedGoal: String = ""

```

```

var body: some View { ZStack { Color("BackgroundColorViews") .ignoresSafeArea()

```

```

VStack { Spacer() Spacer()

```

```

Text("Quanto tempo você \n deseja por dia?") .font(.title2) .fontWeight(.semibold) .padding(.bottom, 30)
.multilineTextAlignment(.center)

```

```

Rectangle() .frame(width: 100, height: 2.5) .foregroundColor(Color("ActionColor")) .padding(.bottom, 30)

```

```

Text("Definir um tempo diário ajuda a manter a consistência. Você pode alterar isso a qualquer
momento") .font(.body) .fontWeight(.light) .padding(.bottom, 30) .multilineTextAlignment(.center)
.foregroundColor(.white)

```

```

MenuSheetPicker( title: "", placeholder: "Selecione seu objetivo diário", selectedValue: $selectedGoal,
options: booksViewModel.goalOptions, formatOption: { "($0) minutos" } )

```

```

Spacer() Spacer()

```

```

Button(action: {

```

```

}) { Text("Iniciar jornada") .font(.title3) .frame(maxWidth: .infinity) .fontWeight(.medium)
.padding(.vertical, 14) .background(Color("ActionColor")) .foregroundColor(.white) .clipShape(Capsule())
} .padding(.bottom, 30)

```

```

} .padding(.horizontal, 26) }

```

```

}}

```

```

#Preview { SecondOnboardView() .environmentObject(BooksViewModel()) } */

```

```

import SwiftUI

```

```
struct SecondOnboardView: View { @EnvironmentObject var booksViewModel: BooksViewModel @State
private var selectedGoal: String = "" var onStart: (() -> Void)? = nil
```

```
var body: some View {
    ZStack {
        Color("BackgroundColorViews")
            .ignoresSafeArea()

        Image("OnboardingOficial2")
            .resizable()
            .scaledToFit()
            .frame(maxWidth: .infinity, maxHeight: .infinity, alignment:
.top)
            .ignoresSafeArea()

        VStack(spacing: 20) {
            Spacer()

            VStack(spacing: 2) {
                Text("Defina a sua ")
                    .font(.largeTitle)
                    .fontWeight(.regular)
                    .foregroundColor(.white) +
                Text("meta")
                    .font(.largeTitle)
                    .fontWeight(.bold)
                    .foregroundColor(.white)

                HStack(spacing: 6) {
                    Text("diária")
                        .font(.bitter(.bold, style: .largeTitle))
                        .foregroundColor(.white)

                    Text("de leitura")
                        .font(.largeTitle)
                        .fontWeight(.regular)
                        .foregroundColor(.white)
                }
            }
            .multilineTextAlignment(.center)
            .padding(.top, 350)

            (Text("Definir um tempo diário ajuda\nem ")
                .foregroundColor(.white.opacity(0.85)) +
                Text("manter a consistência")
                    .foregroundColor(Color("ProgressBar"))
                    .bold())
                .font(.body)
                .multilineTextAlignment(.center)
                .padding(.horizontal, 16)

            MenuSheetPickerOnboarding(
```

```

        title: "",
        placeholder: "Selecione uma meta",
        selectedValue: $selectedGoal,
        options: booksViewModel.goalOptions,
        formatOption: { "\($0) minutos" }
    )
    .padding(.horizontal, 16)
    .padding(.top, 8)

    Text("Você pode alterar isso a qualquer momento")
        .font(.footnote)
        .foregroundColor(.white.opacity(0.5))
        .multilineTextAlignment(.center)

    Spacer()

    ButtonAction(text: "Iniciar jornada", colorButton:
"ActionColor"){
        if let minutes = Int(selectedGoal) {
            booksViewModel.saveDailyGoal(minutes: minutes)
        }
        onStart?()
    }
    .padding(.bottom, 40)
}
.padding(.horizontal, 26)
}
}

```

```

}

```

```

#Preview { SecondOnboardView() .environmentObject(BooksViewModel()) }

```

Arquivo: \./View/Onboarding/SplashView.swift\

```

swift // // SplashView.swift // CH4-Books // // Created by Agatha Barbosa Marinho dos Santos on
20/08/26. //

```

```

import SwiftUI

```

```

struct SplashView: View { var body: some View { ZStack { LinearGradient( gradient: Gradient(stops: [
.init(color: Color(hex: "FEF4C7"), location: 0.00), .init(color: Color(hex: "FEF0B8"), location: 0.05),
.init(color: Color(hex: "FEE78F"), location: 0.20), .init(color: Color(hex: "FEDF6C"), location: 0.35),
.init(color: Color(hex: "FED952"), location: 0.51), .init(color: Color(hex: "FED53F"), location: 0.66),
.init(color: Color(hex: "FED233"), location: 0.83), .init(color: Color(hex: "FFD230"), location: 1.00) ]),
startPoint: .topTrailing, endPoint: .bottomLeading ) .ignoresSafeArea()

```

```

VStack(spacing: 16) {
    Image("FrushLogo")

```

```

        .resizable()
        .scaledToFit()
        .frame(width: 222, height: 184)
        .foregroundColor(Color(red: 0.15, green: 0.10, blue: 0.05))
    }
    .onAppear {
        SoundManager.shared.playSound(named: .splash)
    }
}
}

```

```

}

```

```

extension Color { init(hex: String) { let hex = hex.trimmingCharacters(in:
CharacterSet.alphanumerics.inverted) var int: UInt64 = 0 Scanner(string: hex).scanHexInt64(&int) let r, g,
b: UInt64 switch hex.count { case 6: // RGB (24-bit) (r, g, b) = (int >> 16, int >> 8 & 0xFF, int & 0xFF)
default: (r, g, b) = (1, 1, 0) }
}

```

```

    self.init(
        .sRGB,
        red: Double(r) / 255,
        green: Double(g) / 255,
        blue: Double(b) / 255,
        opacity: 1
    )
}

```

```

}

```

```

#Preview { SplashView() }

```

Arquivo: \./Services/CameraPicker.swift\

```

swift // // CameraPicker.swift // CH4-Books // // Created by Julio Sampaio on 17/08/26. //

```

```

import SwiftUI import UIKit

```

```

struct CameraPicker: UIViewControllerRepresentable {

```

```

    /// Devolve a foto direto para quem chamou, sem precisar de um
    /// `@State` intermediário + `.onChange` (que gerava um redesenho a mais).
    var onImagePicked: (UIImage) -> Void

    @Environment(\.dismiss) private var dismiss

    static var isAvailable: Bool {
        UIImagePickerController.isSourceTypeAvailable(.camera)
    }
}

```

```
}

func makeUIViewController(context: Context) -> UIImagePickerController {
    let picker = UIImagePickerController()
    // No simulador não existe câmera: definir .camera ali causaria crash.
    picker.sourceType = Self.isAvailable ? .camera : .photoLibrary
    picker.delegate = context.coordinator
    return picker
}

func updateUIViewController(_ viewController: UIImagePickerController,
context: Context) { }

func makeCoordinator() -> Coordinator {
    Coordinator(self)
}

final class Coordinator: NSObject, UIImagePickerControllerDelegate,
UINavigationControllerDelegate {
    private let parent: CameraPicker

    init(_ parent: CameraPicker) {
        self.parent = parent
    }

    func imagePickerController(
        _ picker: UIImagePickerController,
        didFinishPickingMediaWithInfo info:
[UIImagePickerController.InfoKey: Any]
    ) {
        if let image = info[.originalImage] as? UIImage {
            parent.onImagePicked(image)
        }
        parent.dismiss()
    }

    func imagePickerControllerDidCancel(_ picker: UIImagePickerController)
{
        parent.dismiss()
    }
}

}
```
